

Unleash Keyless Cloud Deployments with Workload Identity using **OIDC**



 Github Actions

Kosseila Haddalene

CloudThrill



Hi, I'm Kosseila

Cloud Dude



- I live in Toronto 🇧🇪🇨🇦
- ~17 years infra
- Blogger/Speaker
- IaC/DevOps enthusiast
- Founder, @CloudThrill
- Podcaster, TALCO.ca

 [Cloudthrill.ca/blog](https://cloudthrill.ca/blog)

 [Twitter.com/CloudDude_](https://twitter.com/CloudDude_)

 [Linkedin.com/in/kousshd](https://linkedin.com/in/kousshd)

 [GitHub.com/brokedba](https://github.com/brokedba)

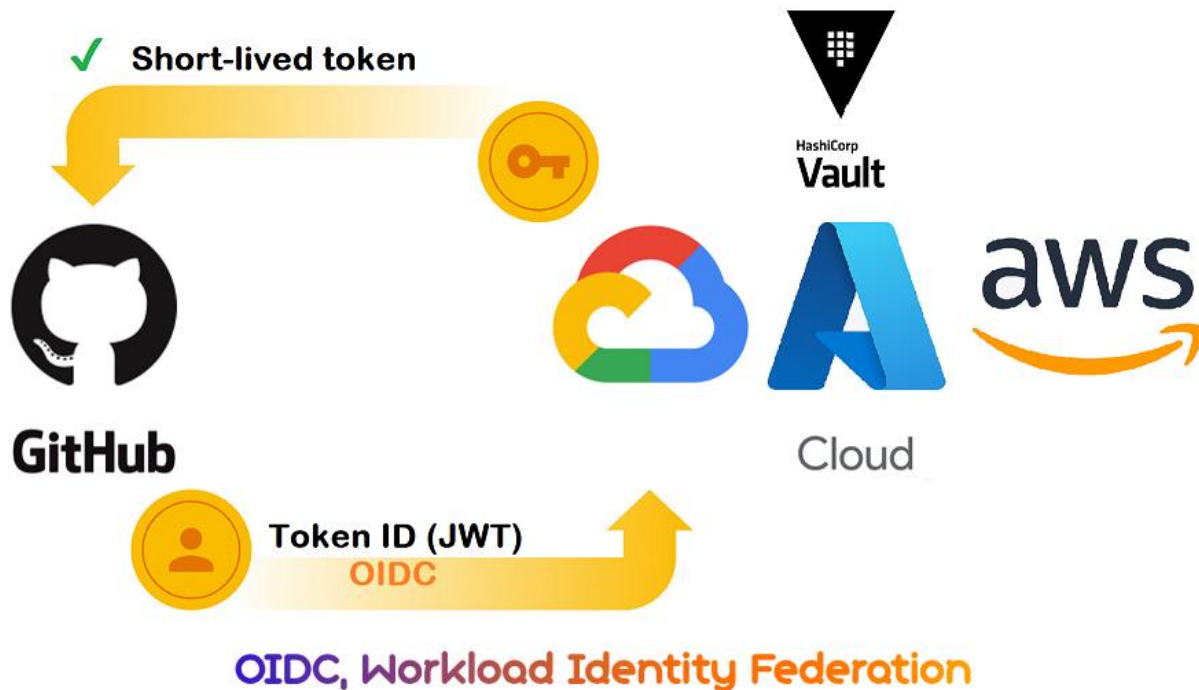
**Tech Beats
Unplugged**



Kosseila Hd Principal Consultant @CloudThrill



“A Safe doorway to your Cloud Platform”





AGENDA

1- Popular Authentications

- ❑ Why is it wrong?

2- OAuth Origins : Context

- ❑ OAuth 2.0 terminology
- ❑ [OAuth 2.0 workflow](#)

3- What is OIDC?

- ❑ OIDC .vs OAuth 2.0
- ❑ OIDC Authentication flow

4- OIDC in GitHub Actions

- ❑ OIDC Authentication flow
- ❑ Multicloud keyless access examples (Azure/ AWS/ GCP)

5- Q&A



Blog post version



SCAN ME



THE STATE OF
SECRETS SPRAWL

2024

How Leaky Was 2023?

12M

secrets occurrences
detected in 2023
(3M unique secrets)

>1 in 10

Commit Authors leaked secrets

7 out of 1K

commits out of 1,000 exposed at least one secret (+50%)

3.7% of repositories active during 2022 leaked a secret

More than 90% of secrets remain valid 5 days after being leaked

12+ million credentials from GitHub were accessible in 2023 alone.

How Leaky Was 2023?

12M

secrets occurrences
detected in 2023
(3M unique secrets)

>1 in 10

Commit Authors leaked secrets

Uber

SAMSUNG

TOYOTA

Infosys



NVIDIA®

7 out of 1K

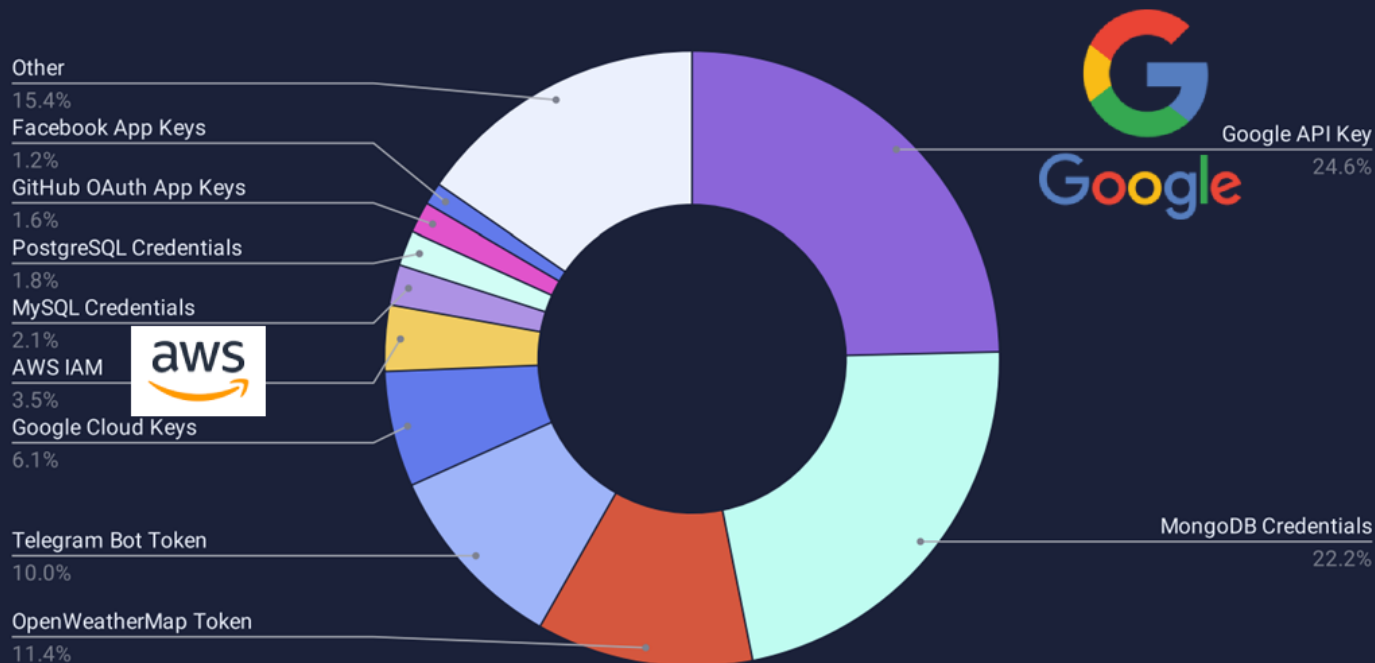
commits out of 1,000 exposed at least one secret (+50%)

3.7% of repositories active during 2022 leaked a secret

More than 90% of secrets remain valid 5 days after being leaked

12+ million credentials from GitHub were accessible in 2023 alone.

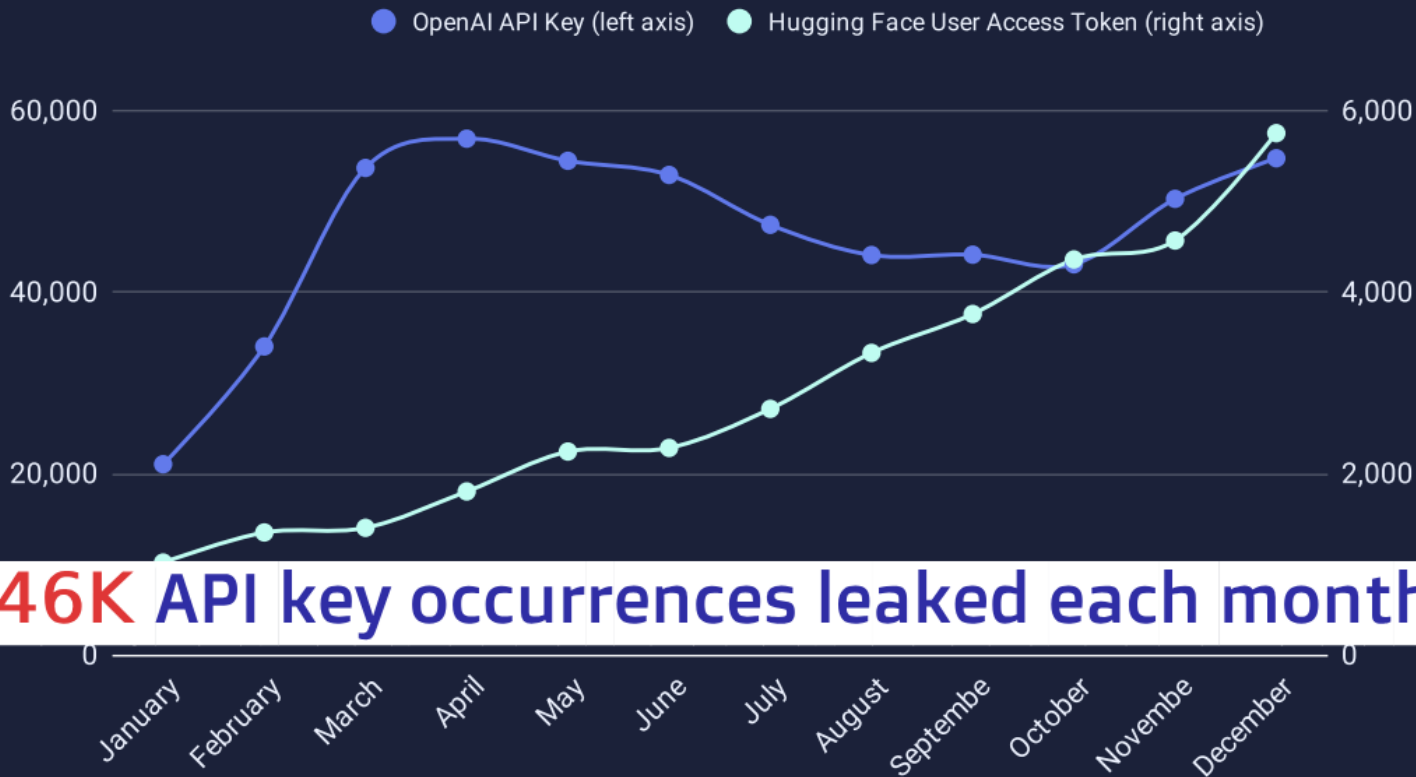
Top 10 Valid Specific Detectors in 2023



- Over 1M Google API secrets
- 250K GCP and 140K AWS IAM Secrets.

12+ million credentials from Github were accessible in 2023 alone.

GenAI Secrets Leaks



Popular Authentications to Cloud Platforms

User or workload principal

❑ User based

Interactive login (GCP, Azure) / access keys (AWS) => "static"

❑ Service principals

GCP (service account key file), Azure (service principal password) => static

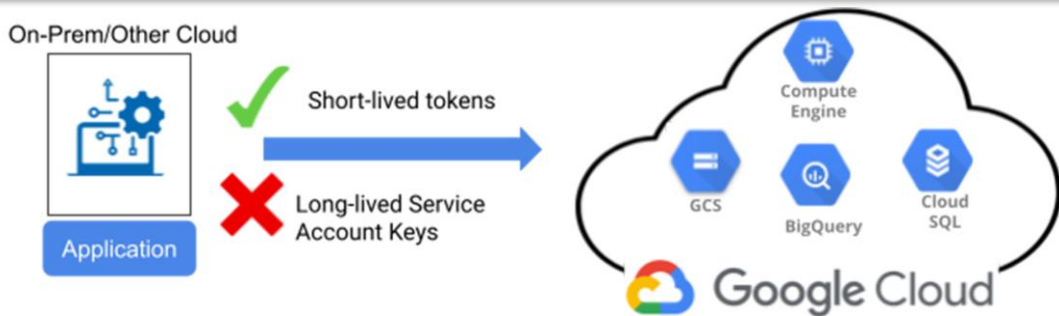
❑ Instance principals/managed identities

Only when Self-hosted runners are located in the cloud => dynamic

❑ Assumed roles

Long-lived secret (i.e. AppRole for Hashicorp Vault) => semi-dynamic

Why is it wrong?



- **Static credentials** never expire maximum vulnerability surface in case of leaks
- **Management overhead:** credential storage/rotation daunting for Ops teams
- **Cloud instance principals** Self hosted runners, is an overkill if you deploy in **MultiCloud**
- **Dynamic** secrets like AppRole (Vault) require static credentials to enable the feature(secret engine)

OAuth `the Underlying Layer of OIDC`

Back in the Stone Age



Step 1
Find Friends

Step 2
Profile Information

Step 3
Profile Picture

Are your friends already on Facebook?

Many of your friends may already be here. Searching your email account is the fastest way to find your friends on Facebook.

Gmail

Your Email:

Email Password:

[Find Friends](#)

Facebook will not store your password.

Yahoo!

[Find Friends](#)

Windows Live Hotmail

[Find Friends](#)

Other Email Service

[Find Friends](#)

facebook.com ~2010

OAuth `the Underlying Layer of OIDC`

Back in the Stone Age



Risks

Step 1
Find Friends

Step 2
Profile Information

Step 3
Profile Picture

Are your friends already on Facebook?

Many of your friends may already be here. Searching your email account is the fastest way to find your friends on Facebook.

 Gmail

Your Email:

Email Password:

Find Friends

 Facebook will not store your password.

 Yahoo!

Find Friends

 Windows Live Hotmail

Find Friends

 Other Email Service

Find Friends

facebook.com ~2010

OAuth `the Underlying Layer of OIDC`

Back in the Stone Age



Risks

1. Impersonation

Step 1
Find Friends

Step 2
Profile Information

Step 3
Profile Picture

Are your friends already on Facebook?

Many of your friends may already be here. Searching your email account is the fastest way to find your friends on Facebook.

Gmail

Your Email:

Email Password:

Find Friends

Facebook will not store your password.

Yahoo!

Find Friends

Windows Live Hotmail

Find Friends

Other Email Service

Find Friends

facebook.com ~2010

OAuth `the Underlying Layer of OIDC`

Back in the Stone Age



Risks

1. Impersonation
2. Phishing Attacks

Step 1
Find Friends

Step 2
Profile Information

Step 3
Profile Picture

Are your friends already on Facebook?

Many of your friends may already be here. Searching your email account is the fastest way to find your friends on Facebook.

 Gmail

Your Email:

Email Password:

Find Friends

 Facebook will not store your password.

 Yahoo!

Find Friends

 Windows Live Hotmail

Find Friends

 Other Email Service

Find Friends

facebook.com ~2010

OAuth `the Underlying Layer of OIDC`

Back in the Stone Age



Risks

1. Impersonation
2. Phishing Attacks
3. App has full scope access

Step 1
Find Friends

Step 2
Profile Information

Step 3
Profile Picture

Are your friends already on Facebook?

Many of your friends may already be here. Searching your email account is the fastest way to find your friends on Facebook.

Gmail

Your Email:

Email Password:

[Find Friends](#)

Facebook will not store your password.

Yahoo!

[Find Friends](#)

Windows Live Hotmail

[Find Friends](#)

Other Email Service

[Find Friends](#)

facebook.com ~2010

OAuth `the Underlying Layer of OIDC`

Back in the Stone Age



Risks

1. Impersonation
2. Phishing Attacks
3. App has full scope access

 Write on **your wall**. Text **your friends**

Step 1
Find Friends

Step 2
Profile Information

Step 3
Profile Picture

Are your friends already on Facebook?

Many of your friends may already be here. Searching your email account is the fastest way to find your friends on Facebook.

 Gmail

Your Email:

Email Password:

Find Friends

 Facebook will not store your password.

 Yahoo!

Find Friends

 Windows Live Hotmail

Find Friends

 Other Email Service

Find Friends

facebook.com ~2010

OAuth 2.0 to the rescue



How to let 3rd party app access user data w/o user passwords ?

- ☐ OAuth eliminates the need for password sharing
- ☐ Allows granular access levels through **scopes**
- ☐ **Apps** receive an authorization key to access target data
- ☐ **Delegated access** with simple user consent

OAuth 2.0 to the rescue



How to let 3rd party app access user data w/o user passwords ?

- ☐ OAuth eliminates the need for password sharing
- ☐ Allows granular access levels through **scopes**
- ☐ **Apps** receive an authorization key to access target data
- ☐ **Delegated access** with simple user consent

Delegation is a process in which an owner authorizes a service provider to perform certain tasks on the owner's behalf.

OAuth 2.0 to the rescue



Q https://accounts.google.com/ServiceLogin?service=mail

Google

Sign in
to continue to Gmail

Email or phone

Forgot email? [Next](#)

Q https://accounts.google.com/ServiceLogin?service=YouTube

Google

Sign in
to continue to YouTube

Email or phone

Forgot email? [Next](#)

Essentially, OAuth is about delegated access.

OAuth 2.0 to the rescue



Q <https://accounts.google.com/ServiceLogin?service=mail>

Google

Sign in
to continue to Gmail

Email or phone

[Forgot email?](#) [Next](#)

Q <https://accounts.google.com/ServiceLogin?service=YouTube>

Google

Sign in
to continue to YouTube

Email or phone

[Forgot email?](#) [Next](#)

Essentially, OAuth is about delegated access.

OAuth 2.0 Terminology



OAuth 2.0 Terminology

OAuth Roles



Resource Owner



Authorization Server



Client Application



Resource Server

OAuth 2.0 Terminology

OAuth Roles

1



YOU!



Resource Owner



Authorization Server



Client Application



Resource Server

OAuth 2.0 Terminology

OAuth Roles

1



YOU!



Resource Owner



Authorization Server



Client Application

2



Resource Server

OAuth 2.0 Terminology

OAuth Roles

1



YOU!



Resource Owner

3



Authorization Server



✓ ACCEPT

✗ REJECT



Client Application

2



Resource Server

OAuth 2.0 Terminology

OAuth Roles

1



YOU!



Resource Owner

3



Authorization Server



✓ ACCEPT

✗ REJECT



Client Application

2



Resource Server

4

Alibaba Cloud



OAuth 2.0 Terminology

OAuth Roles

1



YOU!



Resource Owner



Client Application

2



3

Authorization Server

Often:
Same
location



Resource Server

4



✓ ACCEPT

✗ REJECT

Alibaba Cloud

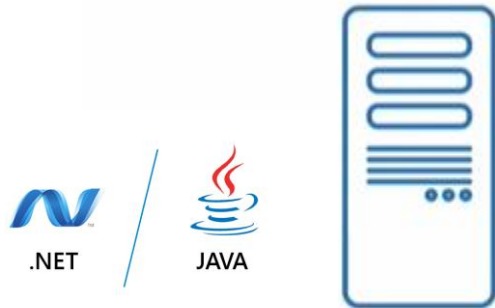


IBM Cloud

ORACLE
CLOUD

OAuth 2.0 Clients

Confidential Clients



Application running on a server

Has the ability to keep strings secret
since code is running in a trusted environment

Public Clients

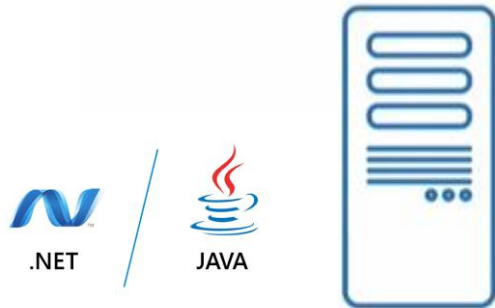


The application can't keep strings secret

JavaScript/Single-Page apps: "view source"
Native apps: decompile and extract strings

OAuth 2.0 Clients

Confidential Clients



Application running on a server

Has the ability to keep strings secret since code is running in a trusted environment

Public Clients

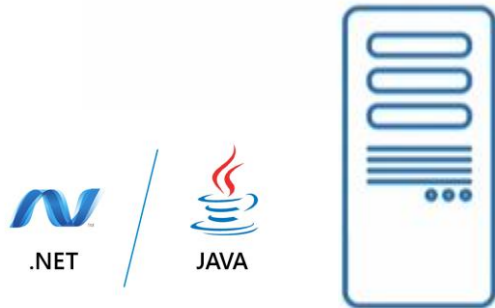


The application can't keep strings secret

JavaScript/Single-Page apps: "view source"
Native apps: decompile and extract strings

OAuth 2.0 Clients

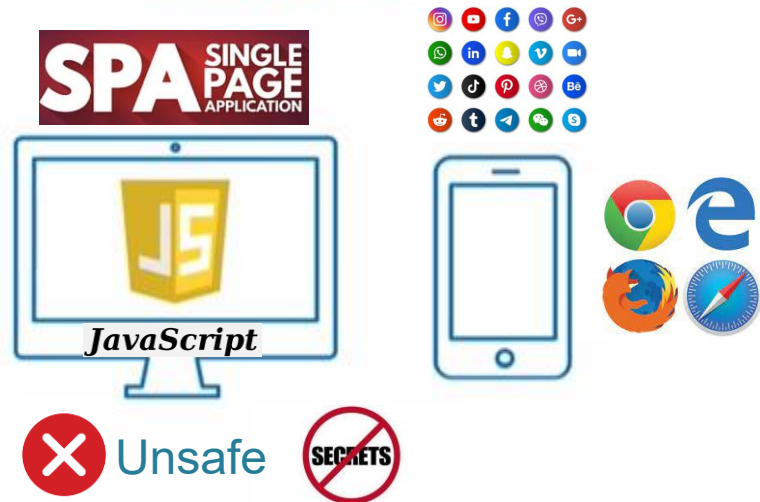
Confidential Clients



Application running on a server

Has the ability to keep strings secret
since code is running in a trusted environment

Public Clients

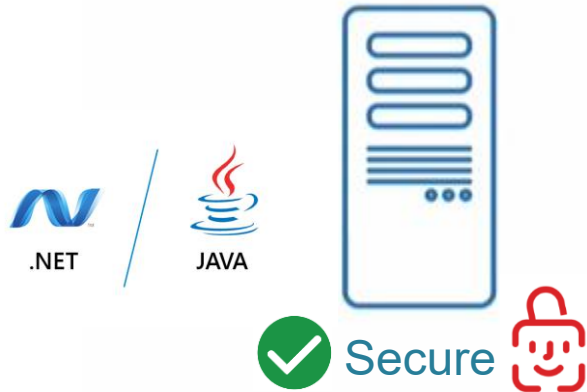


The application can't keep strings secret

JavaScript/Single-Page apps: "view source"
Native apps: decompile and extract strings

OAuth 2.0 Clients

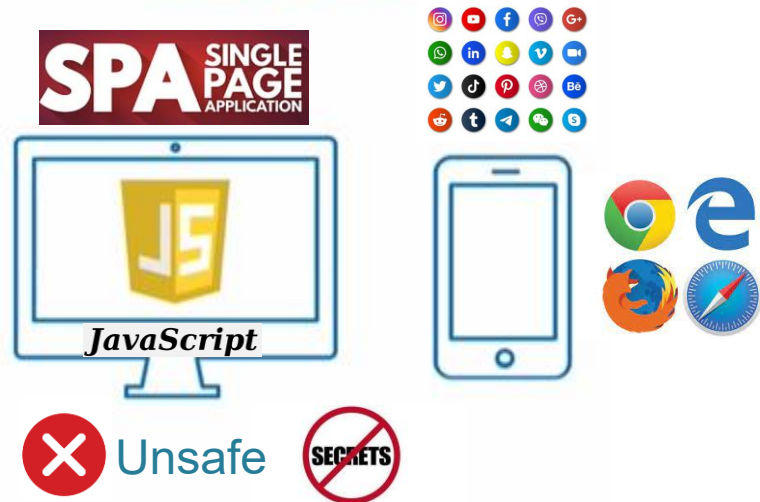
Confidential Clients



Application running on a server

Has the ability to keep strings secret
since code is running in a trusted environment

Public Clients



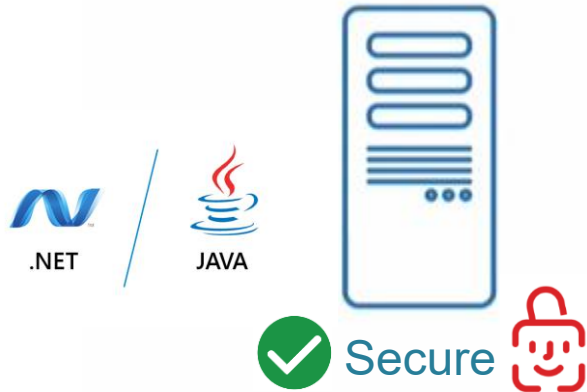
The application can't keep strings secret

JavaScript/Single-Page apps: "view source"
Native apps: decompile and extract strings

OAuth 2.0 Clients

Confidential Clients

* Back channel



Application running on a server

Has the ability to keep strings secret
since code is running in a trusted environment

Public Clients

* Front channel



The application can't keep strings secret

JavaScript/Single-Page apps: "view source"
Native apps: decompile and extract strings

OAuth 2.0 Terminology

Application registration (Day1)

- Authorization Server registers the **Client** (app)
- Established a future working relationship.

Register a new OAuth application

Application name

Something users will recognize and trust

Homepage URL

The full URL to your application homepage

Application description

This is displayed to all users of your application

Authorization callback URL

Your application's callback URL. Read our [OAuth documentation](#) for more information.

Register application

[Cancel](#)

OAuth 2.0 Terminology

Application registration (Day1)

- Authorization Server registers the **Client** (app)
- Established a future working relationship.
 1. 🖊️ Set **Redirect URI /Callback URI**

Register a new OAuth application

Application name

Something users will recognize and trust

Homepage URL

The full URL to your application homepage

Application description

Application description is optional

Redirect a **Resource Owner** back to after consent.



Authorization callback URL

Your application's callback URL. Read our [OAuth documentation](#) for more information.

Register application

Cancel

OAuth 2.0 Terminology

Application registration (Day1)

- Authorization Server registers the **Client** (app)
- Established a future working relationship.
 1. 🖊️ Set **Redirect URI /Callback URI**
 2. Authorization Server Generates **Client ID** + **Client Secret**

Register a new OAuth application

Application name

Something users will recognize and trust

Homepage URL

The full URL to your application homepage

Application description

Application description is optional

Redirect a **Resource Owner** back to after consent.



Authorization callback URL

Your application's callback URL. Read our [OAuth documentation](#) for more information.

Register application

Cancel

OAuth 2.0 Terminology

Application registration (Day1)

- Authorization Server registers the **Client** (app)
- Established a future working relationship.

1.  Set **Redirect URI /Callback URI**

2. Authorization Server Generates

Client ID + **Client Secret**



Identify **Client** with **Auth Server**

Register a new OAuth application

Application name

Something users will recognize and trust

Homepage URL

The full URL to your application homepage

Application description

Application description is optional

Redirect a **Resource Owner** back to after consent.

Authorization callback URL

Your application's callback URL. Read our [OAuth documentation](#) for more information.

Register application

Cancel

OAuth 2.0 Terminology

Application registration (Day1)

- Authorization Server registers the **Client** (app)
- Established a future working relationship.

1.  Set **Redirect URI /Callback URI**

2. Authorization Server Generates

Client ID + **Client Secret**



Identify **Client** with **Auth Server**



Password given to the **Client** for private exchange

Register a new OAuth application

Application name

Something users will recognize and trust

Homepage URL

The full URL to your application homepage

Application description

Application description is optional

Redirect a **Resource Owner** back to after consent.

Authorization callback URL

Your application's callback URL. Read our [OAuth documentation](#) for more information.

Register application

Cancel

OAuth 2.0 Terminology

- Scope
- Grant Type
- Consent
- Authorization Code
- State
- Access Token/ Refresh Token

OAuth 2.0 Terminology

- **Scope**

Type of permission the Client wants, such as access to data or to perform actions

- **Grant Type**

- **Consent**

- **Authorization Code**

- **State**

- **Access Token/ Refresh Token**

OAuth 2.0 Terminology

- **Scope**

Type of permission the Client wants, such as access to data or to perform actions

- **Grant Type**

{Code} **Authorization Code**, a Client expects from the **auth server** for future access request

- **Consent**

- **Authorization Code**

- **State**

- **Access Token/ Refresh Token**

OAuth 2.0 Terminology

- **Scope**

Type of permission the Client wants, such as access to data or to perform actions

- **Grant Type**

{Code} **Authorization Code**, a Client expects from the **auth server** for future access request

- **Consent**

Given from the **Owner** to the **Client** (by informing the **Authorization Server**)

- **Authorization Code**

- **State**

- **Access Token/ Refresh Token**

OAuth 2.0 Terminology

- **Scope**

Type of permission the Client wants, such as access to data or to perform actions

- **Grant Type**

{Code} **Authorization Code**, a Client expects from the **auth server** for future access request

- **Consent**

Given from the **Owner** to the **Client** (by informing the **Authorization Server**)

- **Authorization Code**

Grant code, the Client exchanges with the Authorization Server for an Access Token

- **State**

- **Access Token/ Refresh Token**

OAuth 2.0 Terminology

- **Scope**

Type of permission the Client wants, such as access to data or to perform actions

- **Grant Type**

{Code} **Authorization Code**, a Client expects from the **auth server** for future access request

- **Consent**

Given from the **Owner** to the **Client** (by informing the **Authorization Server**)

- **Authorization Code**

Grant code, the Client exchanges with the Authorization Server for an Access Token

- **State**

Random string generated by the app, allowing clients to confirm the Auth server's reply is legit

- **Access Token/ Refresh Token**

OAuth 2.0 Terminology

- **Scope**

Type of permission the Client wants, such as access to data or to perform actions

- **Grant Type**

{Code} **Authorization Code**, a Client expects from the **auth server** for future access request

- **Consent**

Given from the **Owner** to the **Client** (by informing the **Authorization Server**)

- **Authorization Code**

Grant code, the Client exchanges with the Authorization Server for an Access Token

- **State**

Random string generated by the app, allowing clients to confirm the Auth server's reply is legit

- **Access Token/ Refresh Token**

Short-lived key, the Client **obtains/refreshes** from the **auth server** to use target resources

OAuth 2.0 Terminology

Payload

- **Client** Exchanges the authorization code for an **access token**.



```
POST https://authorization-server.com/token
```

```
grant_type=authorization_code
```

```
&client_id=Aq4CBzi_fJlU0DyUfqf3bJM1
```

```
&client_secret=cmeXxk0QXsavfJoHmiJfd6XuoeozN-fneRshvDgc0yYIMfrk
```

```
&redirect_uri=https://www.oauth.com/playground/authorization-code.html
```

```
&code=aIiR4IpJzxALhoDGLKoW97em7GocFZTlSrhcDMex-EWyQaf
```

OAuth 2.0 Terminology

Payload

- **Client** Exchanges the authorization code for an **access token**.



```
POST https://authorization-server.com/token
```



```
grant_type=authorization_code
```

```
&client_id=Aq4CBzi_fJlU0DyUfqf3bJM1
```

```
&client_secret=cmeXxk0QXsavfJoHmiJfd6XuoeozN-fneRshvDgc0yYIMfrk
```

```
&redirect_uri=https://www.oauth.com/playground/authorization-code.html
```



```
&code=aIiR4IpJzxALhoDGLKoW97em7GocFZTlSrhcDMex-EWyQaf
```

OAuth 2.0 Terminology

Payload

- **Client** Exchanges the authorization code for an **access token**.



```
{
  "token_type": "Bearer",
  "expires_in": 86400,
  "access_token": "v_zhG1tfKrtmWNJLqex0XgeyJANfDGLJRsy",
  "scope": "photo offline_access",
  "refresh_token": "NfBjI2F15i0WDU2T0r59BZ4J"
}
```

OAuth 2.0 Terminology

Payload

- **Client** Exchanges the authorization code for an **access token**.



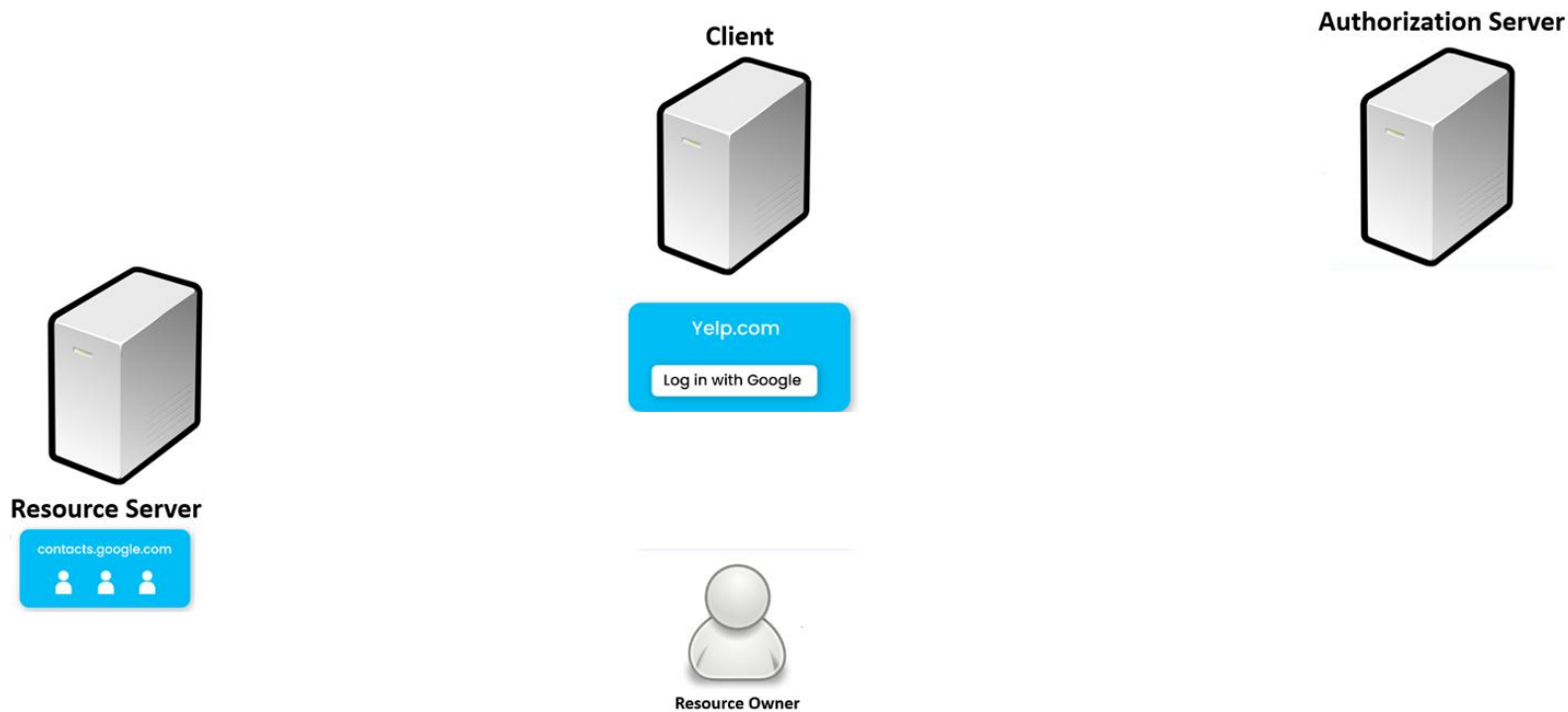
```
{
  "token_type": "Bearer",
  "expires_in": 86400,
  "access_token": "v_zhG1tfKrtmWNJLqexOXgeyJANfDGLJRsy",
  "scope": "photo offline_access",
  "refresh_token": "NfBjI2F15i0WDU2T0r59BZ4J"
}
```

Try it !

 <https://www.oauth.com/playground>

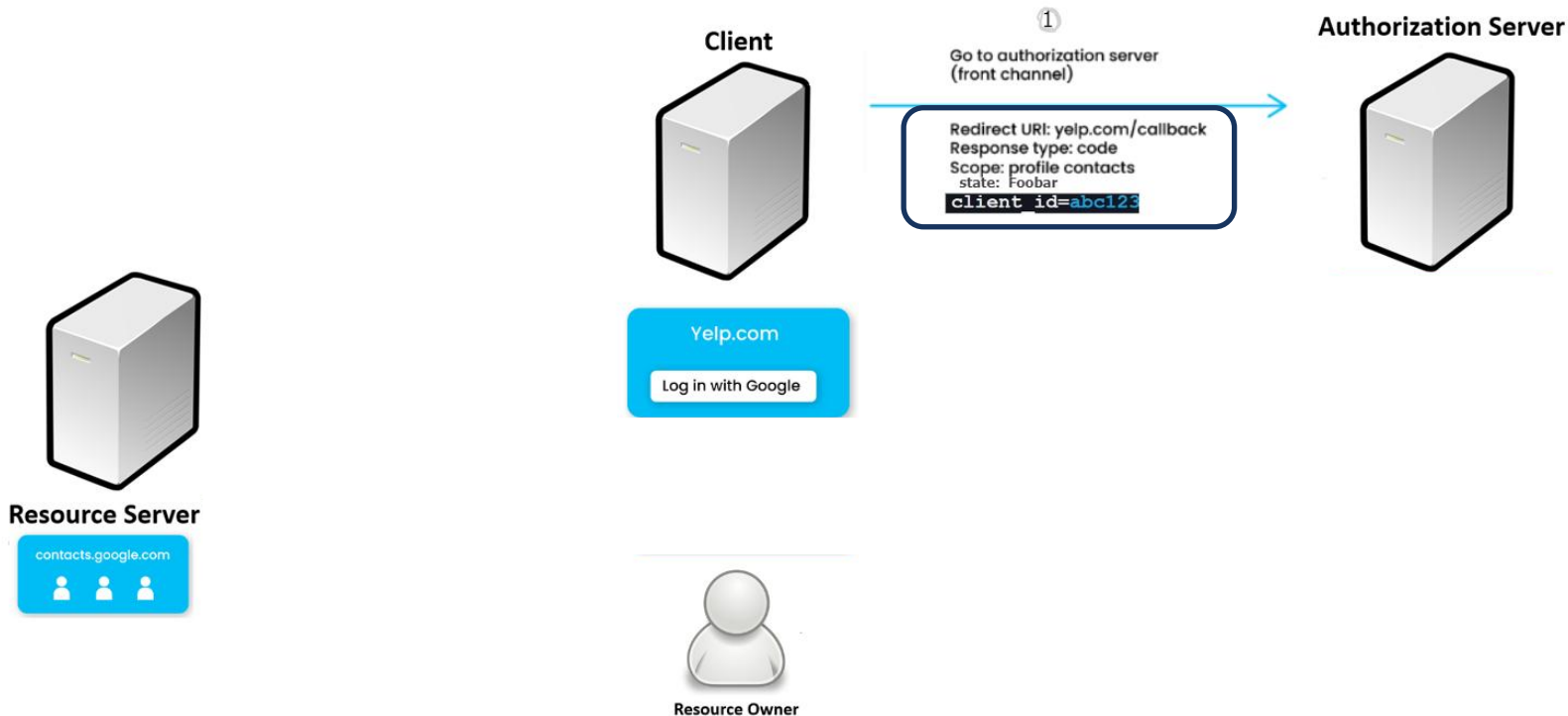
OAuth 2.0 Workflow

OAuth 2.0 Authorization flow



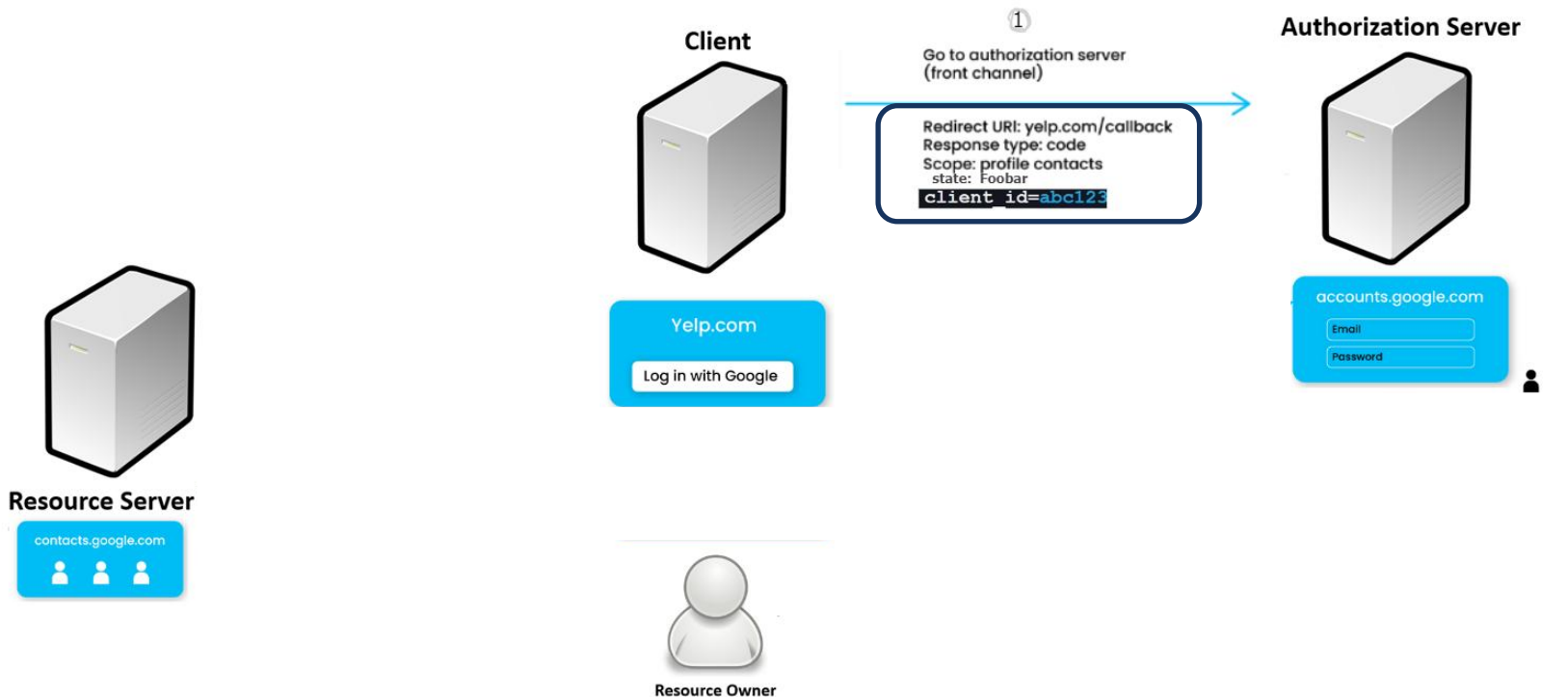
OAuth 2.0 Workflow

OAuth 2.0 Authorization flow



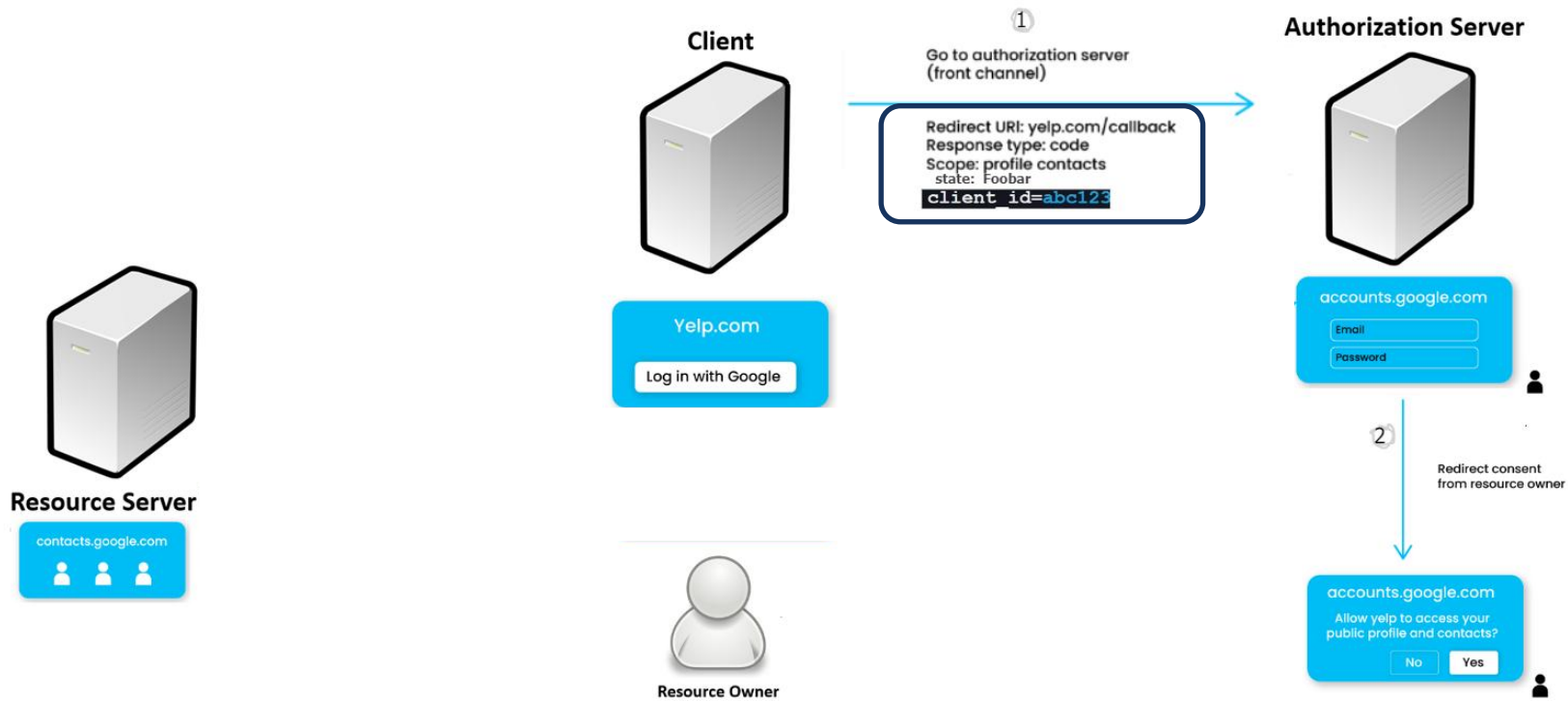
OAuth 2.0 Workflow

OAuth 2.0 Authorization flow



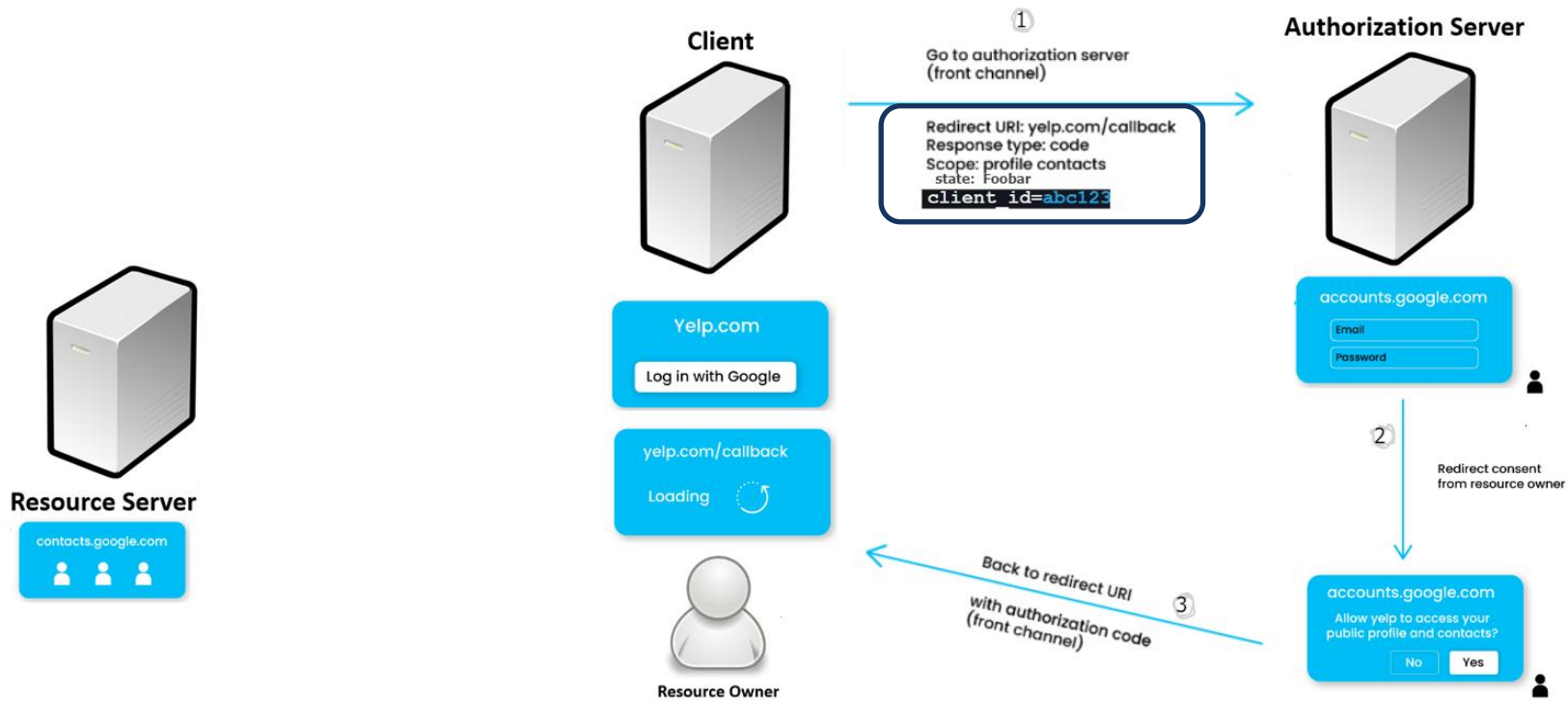
OAuth 2.0 Workflow

OAuth 2.0 Authorization flow



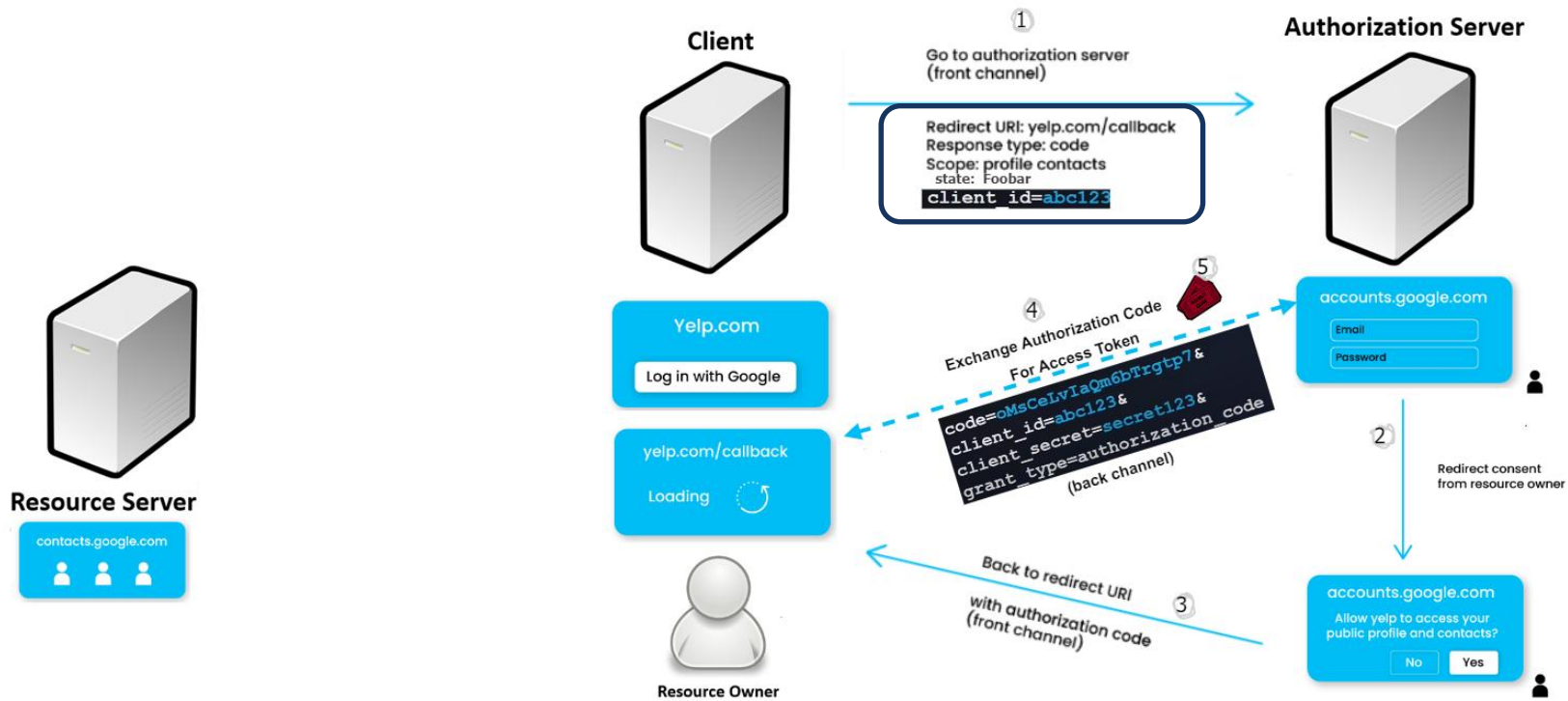
OAuth 2.0 Workflow

OAuth 2.0 Authorization flow



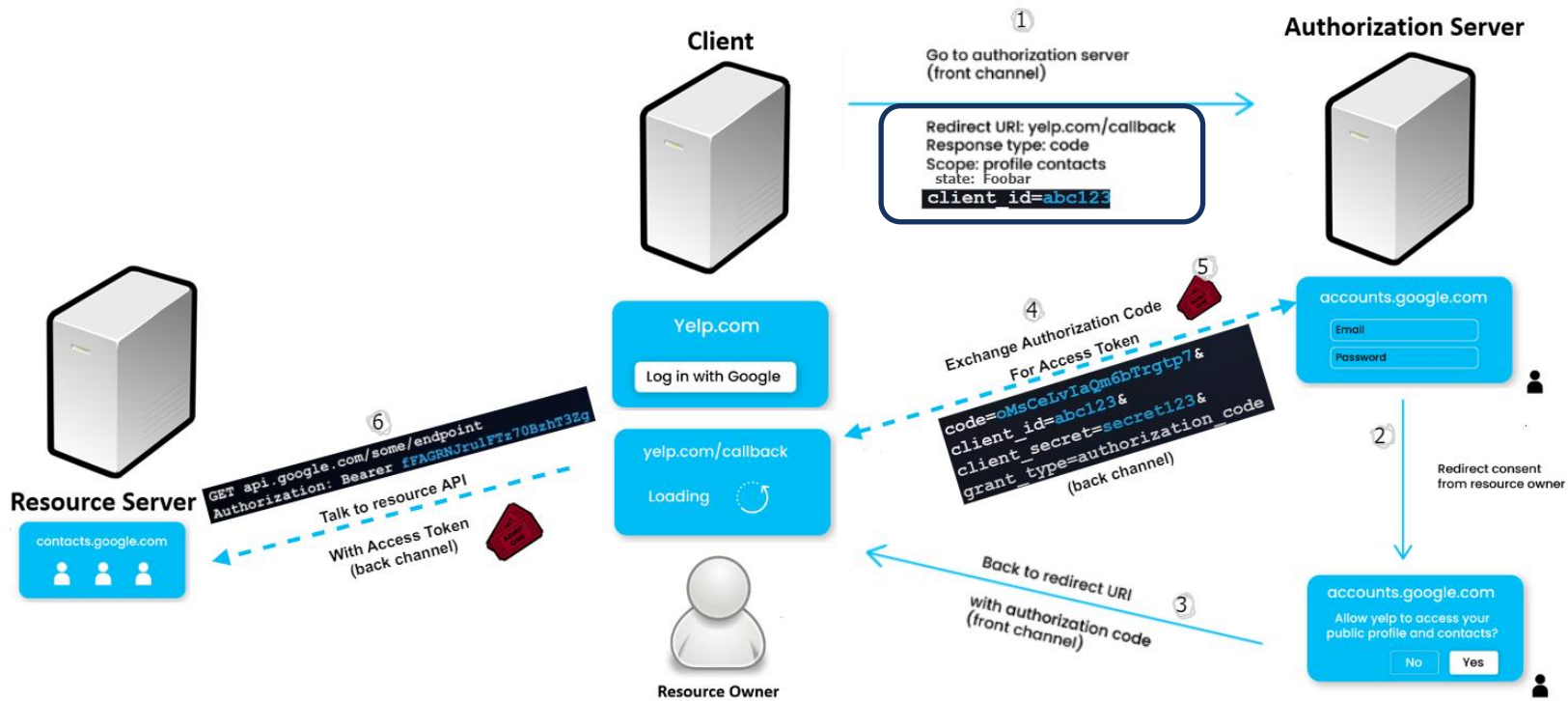
OAuth 2.0 Workflow

OAuth 2.0 Authorization flow



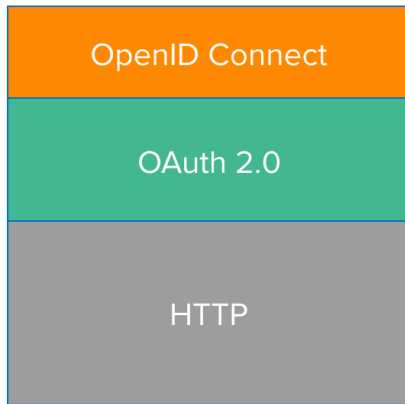
OAuth 2.0 Workflow

OAuth 2.0 Authorization flow



What is OIDC?

Open ID Connect (AuthN)



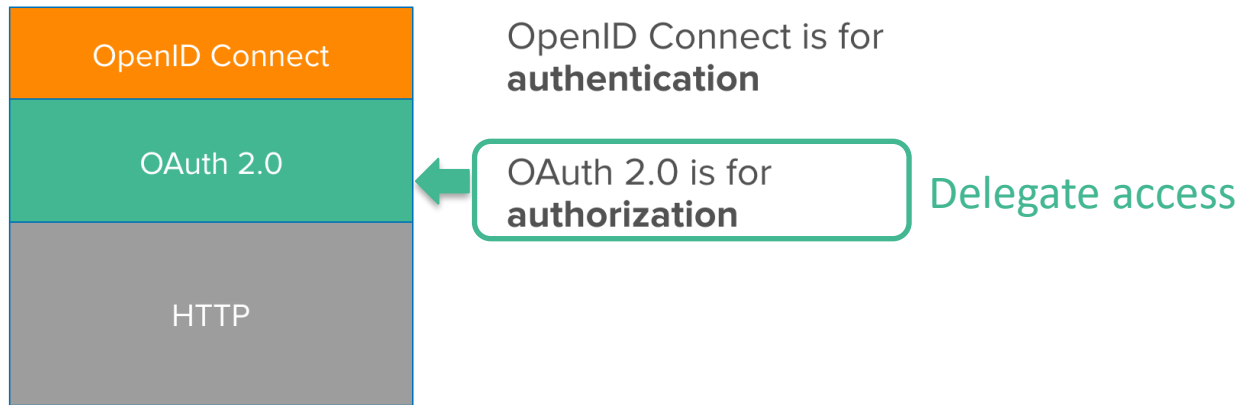
OpenID Connect is for
authentication

OAuth 2.0 is for
authorization

- Authentication protocol built on top of **OAuth2.0** that adds an **ID layer** on top of it
- Provides user information
- Enables clients to establish login sessions.
- **OAuth 2.0** provides Access delegation via scoped access tokens
- **OIDC** provides user **authentication** and **SSO** on top of OAuth's Authorization.

What is OIDC?

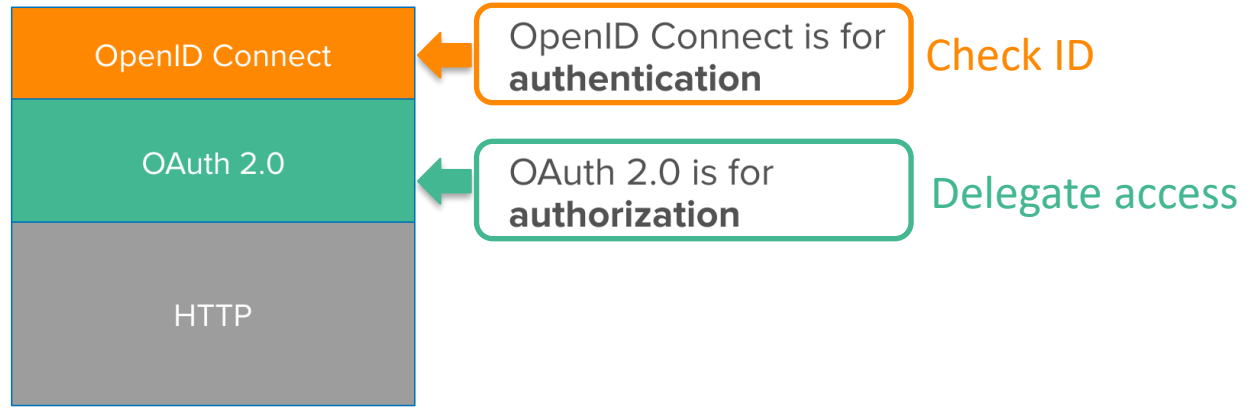
Open ID Connect (AuthN)



- Authentication protocol built on top of **OAuth2.0** that adds an **ID layer** on top of it
- Provides user information
- Enables clients to establish login sessions.
- **OAuth 2.0** provides Access delegation via scoped access tokens
- **OIDC** provides user **authentication** and **SSO** on top of OAuth's Authorization.

What is OIDC?

Open ID Connect (AuthN)



- Authentication protocol built on top of **OAuth2.0** that adds an **ID layer** on top of it
- Provides user information
- Enables clients to establish login sessions.
- **OAuth 2.0** provides Access delegation via scoped access tokens
- **OIDC** provides user **authentication** and **SSO** on top of OAuth's Authorization.

What is OIDC?

Open ID Connect (AuthN)



Gives the application a way to make API requests

Accessing APIs



Tells the application about the user (ID, name, email, etc)

Identification

- Authentication protocol built on top of **OAuth2.0** that adds an **ID layer** on top of it
- Provides user information
- Enables clients to establish login sessions.
- **OAuth 2.0** provides Access delegation via scoped access tokens
- **OIDC** provides user [authentication](#) and [SSO](#) on top of OAuth's Authorization.

OIDC addons

- **User Info endpoint** (ID, name, email, ..)
- **ID Token**
 - ❑ Contains information about the owner who signed
 - ❑ Encoded using **JWT** (Jason Web Token) format, which is readable by the client.
 - ❑ The data inside the ID Token is called => **claims**.
 - ❑ In a OIDC flow, the client requests an **ID token** along with an **access token**.
- **Set of scopes and standardized implementation**

OIDC addons

ID token (JWT)

```
eyJhbGciOiJIUzI1NiIsImtpZCI6IkpzRU1EYnh0Y25xZWVJISVhuc2UubWl3UUpfWF9rZWmJyaEtBMGMifQ
```

```
eyJzdWIiOiIwMHU5bzFuaWtqdG9CZzVabzBoNyIsInZlciI6MSwiaXNzIjoiaHR0cHM6Ly9kZXltMzQxNjA3Lm9rdGFwcmV2aWV3LmNvbS9vYXV0aDIvYXVzOW84d3ZraG9ja3c5VEwwaDciLCJhdWQiOiJzWFNlbkx4eFBpOGtRVmpKRTVzNCIsImldCI6MTUwOTA0Tg5OCwiZXhwIjoxNTA5MDUzNDk4LCJqdGkiOiJRC5oa2RXSXNBSXZTbnBGYVFHTVRYUGNVSmhhMkgwS2c5Yk13ZEVvVm1ZZHN3IiwiaW1yIjpbImtiYSIsIm1mYSIsInB3ZCJdLCJpZHAiOiIwMG85bzFuaWprYWpLeGNpbjBoNyIsIm5vbmNI1IjoiaWpwMmFzeHlqN2UiLCJhdXRoX3RpbWUiOiJlMDkwNDk3MTI9
```

```
dv4Ek8B4BDee1PcQT_4zm7kxDEY1sRIGbLoNtlodZcSzHz-XU5GkKy16sAVmdX0IPU1AIrJAhnFQWQ-XZLBVPjETiZE8CgNg5uqNmeXMUnYnQmvN5ow1XUZ8Gcub-GAbJ8-NQuyBmyec1j3gmGzX3wemke8NkuI6SX2L4Wj1PyvknBtbjfiF9ud1-ERKbobaFbnjDFOFTzvL6g34SpMmZwy6uc_Hs--n4IC-ex_Ps3FcMwRggCW_-7o2FpH6rJTOGPZYrOx44n3ZwAu2dGm6axtPI-sqU8b6sw7DaHpogD_hxsXgMIOzOBMBYsQEiczoGn71ZFz_107FiW4dH6g
```

Header

Payload
(claims)

Signature



The ID token (JWT)

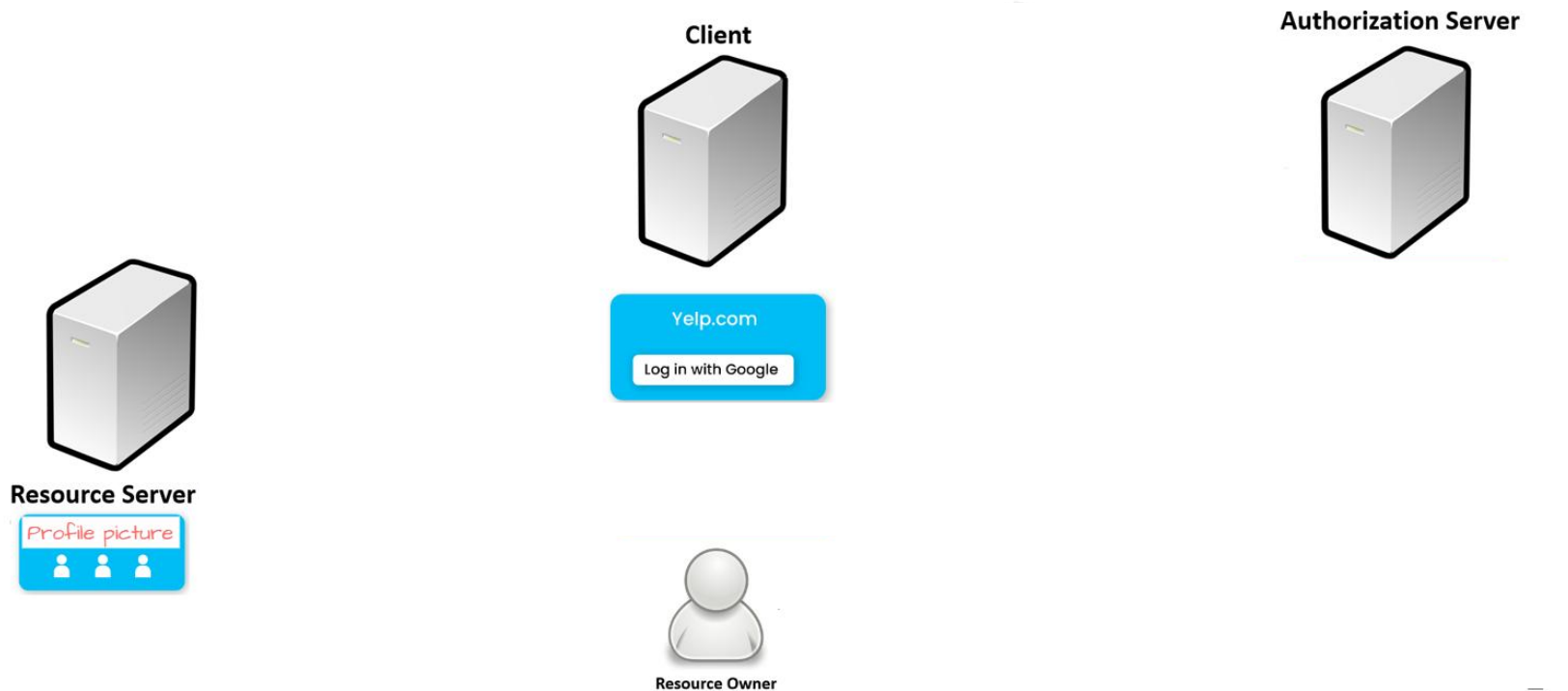
(Header)

```
{
  "iss": "https://accounts.google.com",
  "sub": "you@gmail.com",
  "name": "John Doe",
  "aud": "s6BhdRkqt3",
  "exp": 1311281970,
  "iat": 1311280970,
  "auth_time": 1311280969,
}
```

(Signature)

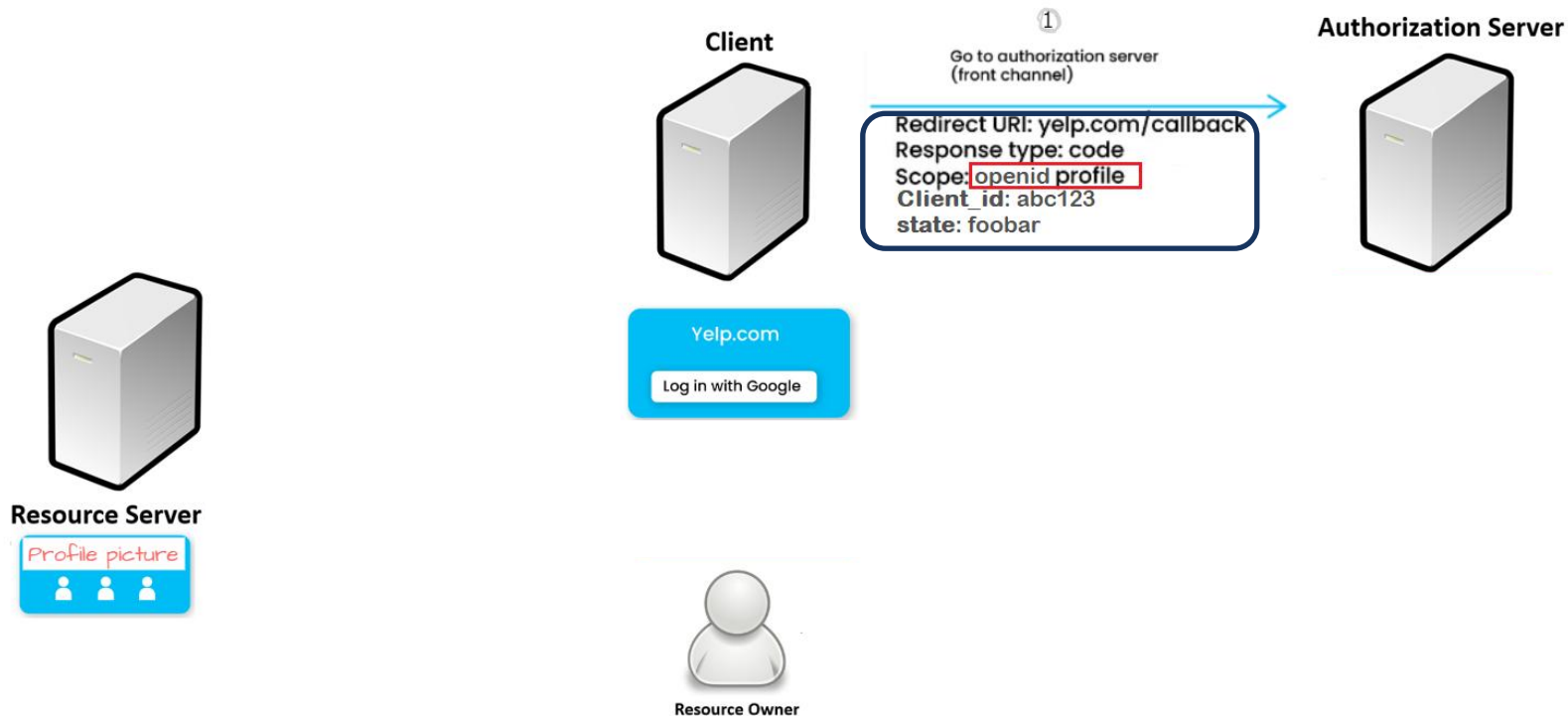
OIDC Workflow

OpenID connect flow



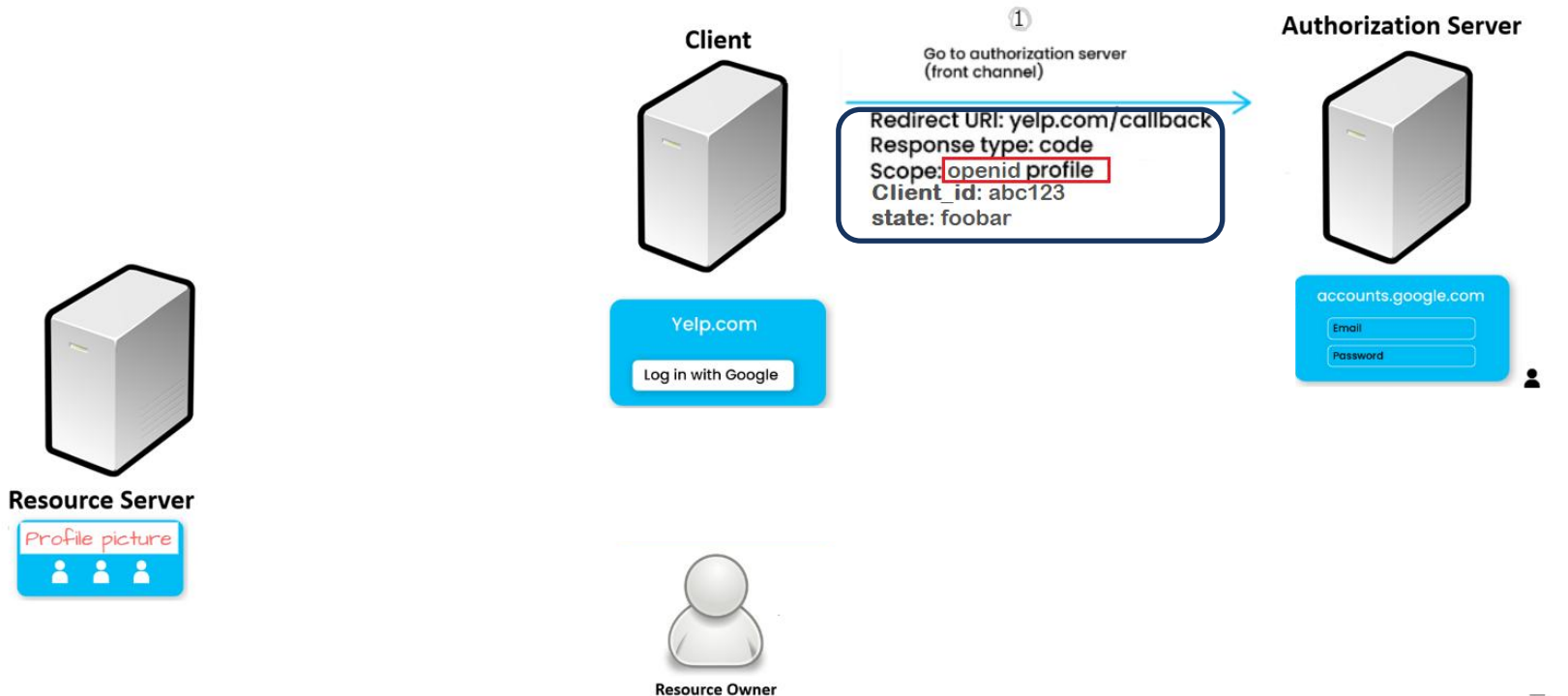
OIDC Workflow

OpenID connect flow



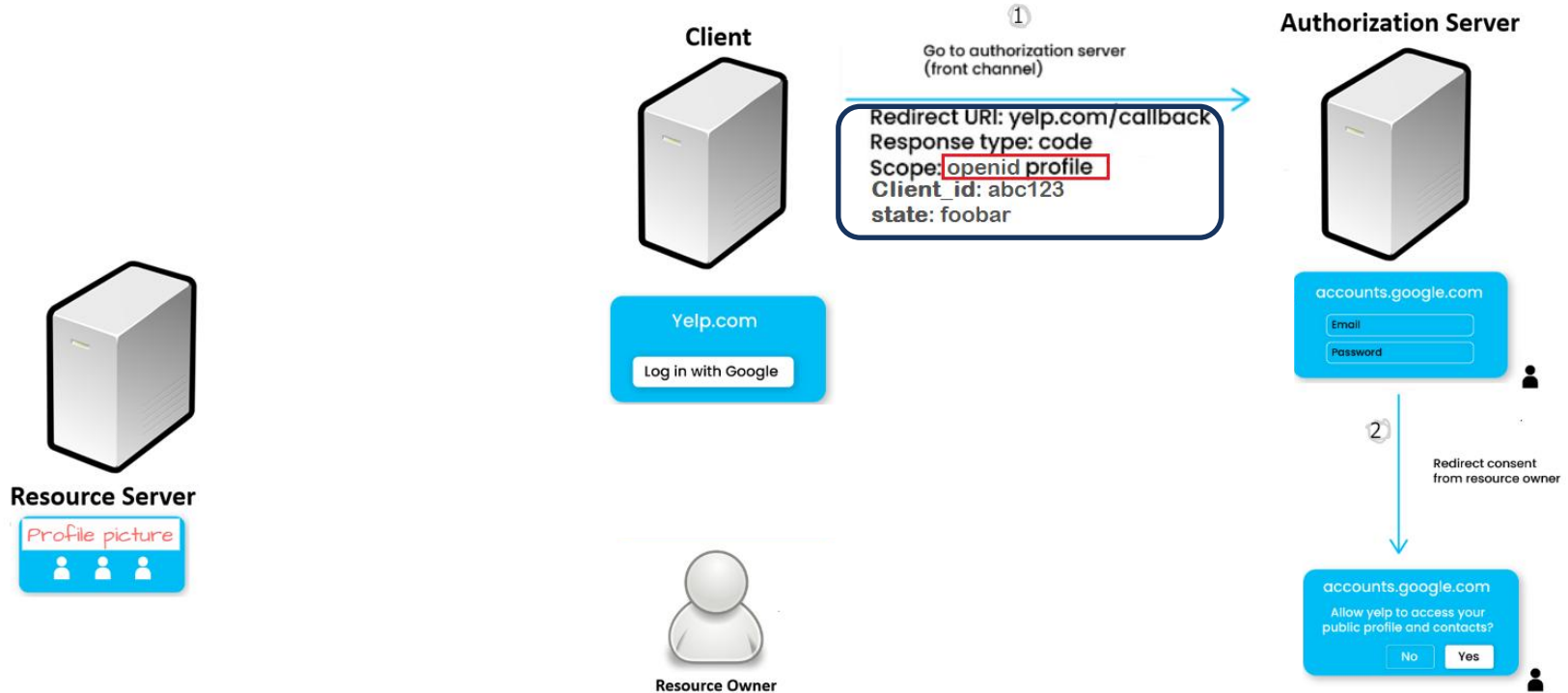
OIDC Workflow

OpenID connect flow



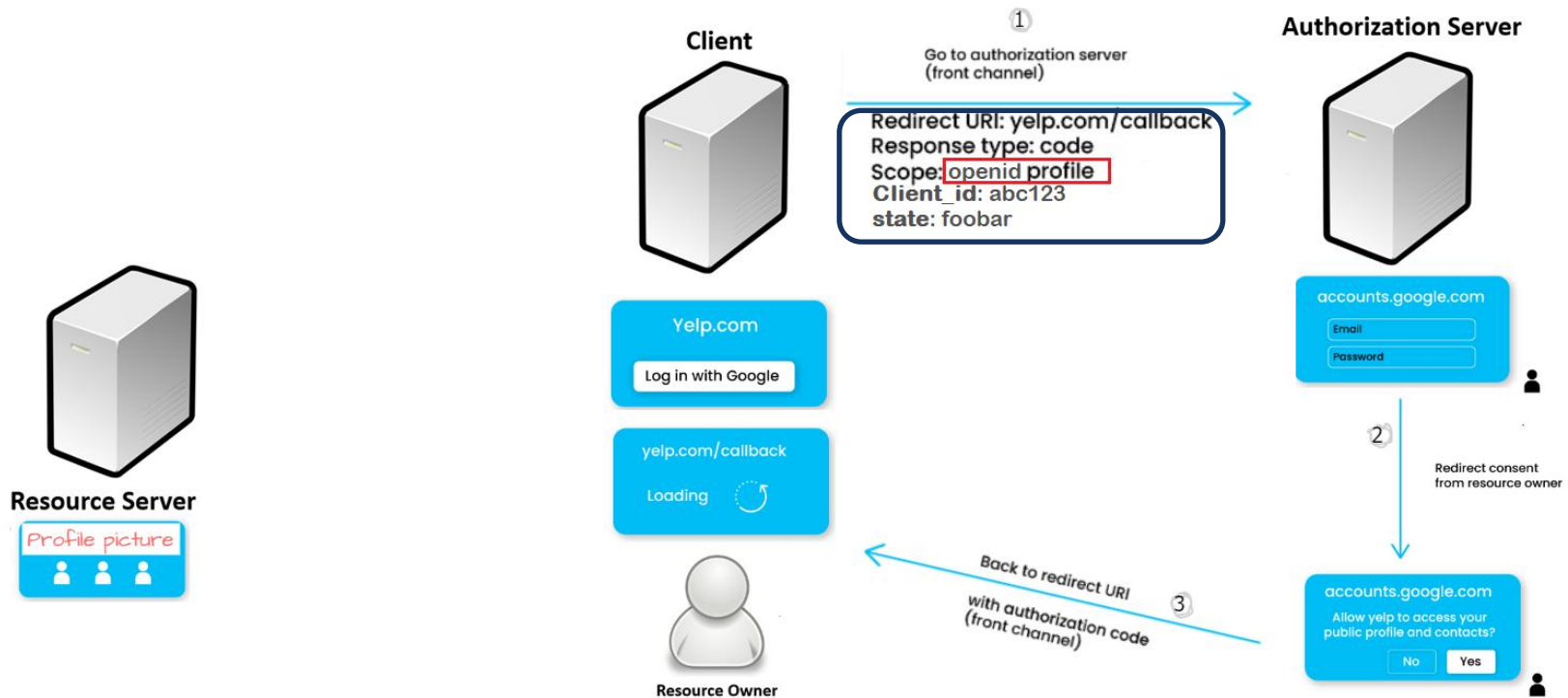
OIDC Workflow

OpenID connect flow



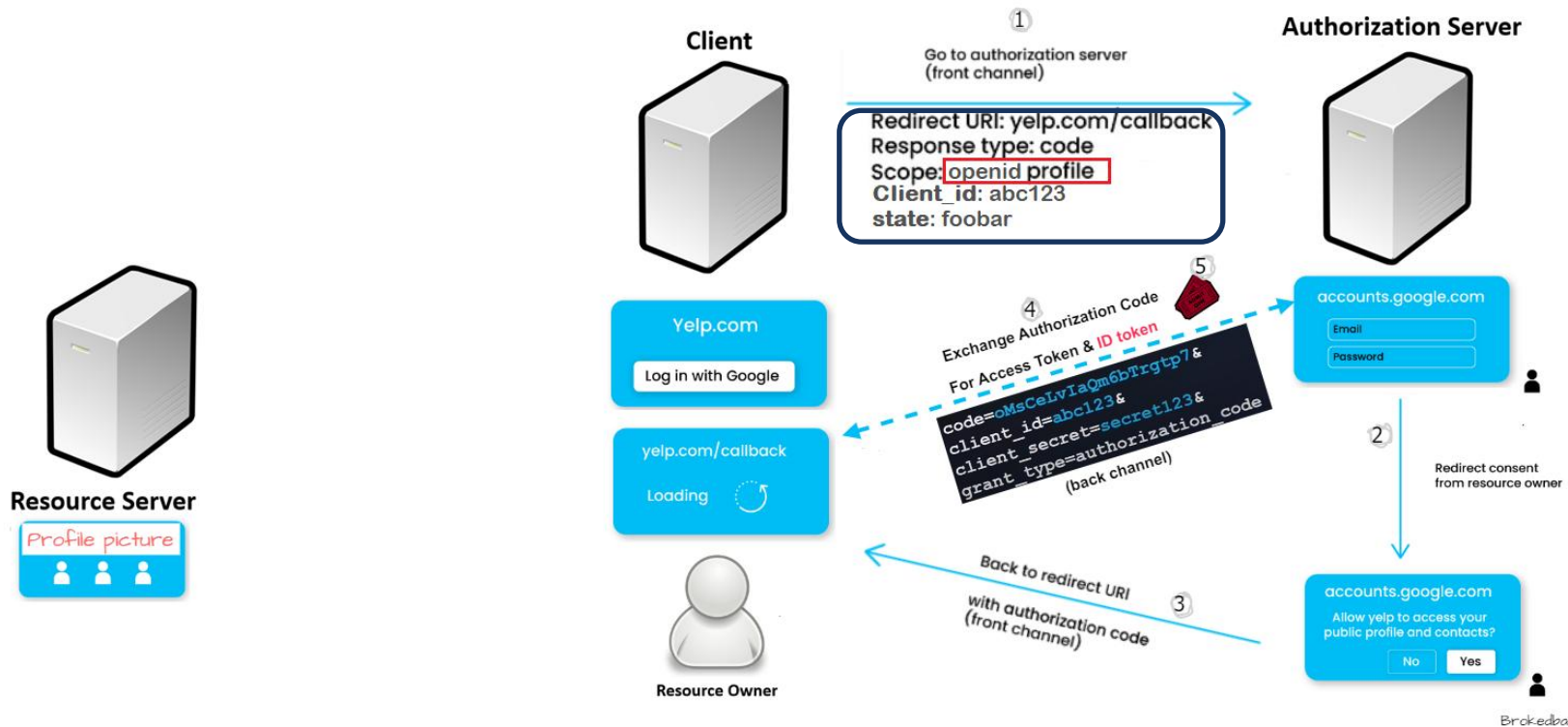
OIDC Workflow

OpenID connect flow



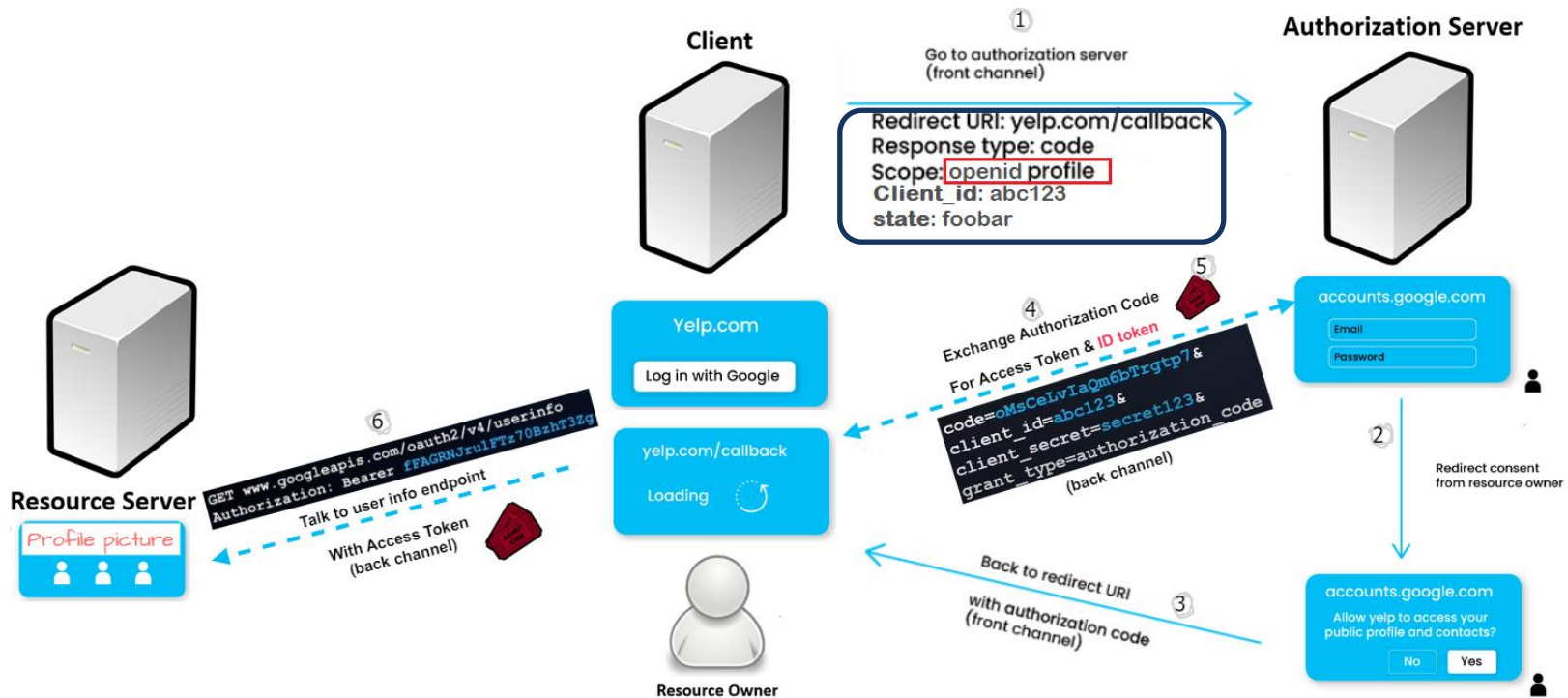
OIDC Workflow

OpenID connect flow



OIDC Workflow

OpenID connect flow



OIDC in GitHub Actions

- Allows GitHub Actions workflows to access Cloud resources with temporary tokens

Benefits:

- ❑ Seamless authentication with cloud providers with **0 secrets** stored
- ❑ No duplication of secret management (GitHub and Cloud)
- ❑ Enables admins to rely on provider's IAM for least privilege access

OIDC in GitHub Actions

- Allows GitHub Actions workflows to access Cloud resources with temporary tokens

Benefits:

- ❑ Seamless authentication with cloud providers with **0 secrets** stored
- ❑ No duplication of secret management (GitHub and Cloud)
- ❑ Enables admins to rely on provider's IAM for least privilege access

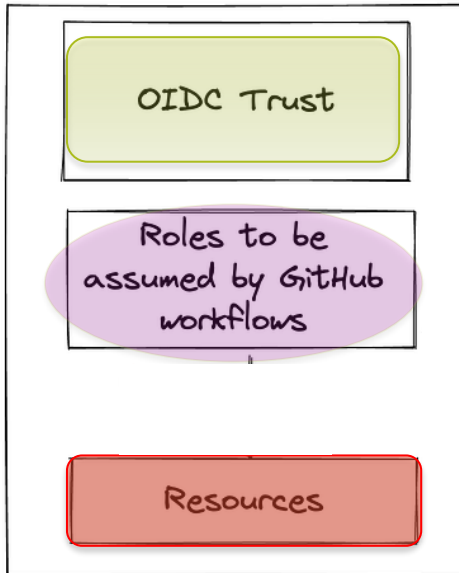
💡 Using **OIDC** + **Workload Identity Federation** ➡ max security + min maintenance
(Rotation, management,...)

OIDC in GitHub Actions (Workflow)

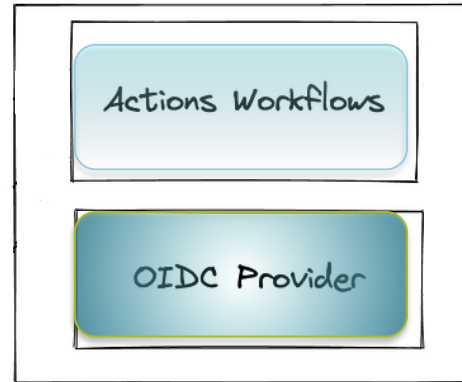
- **App Registration:** Set up the OIDC trust in your cloud provider as a federated identity, then update your workflow to authenticate using ID tokens.

OIDC in GitHub Actions (Workflow)

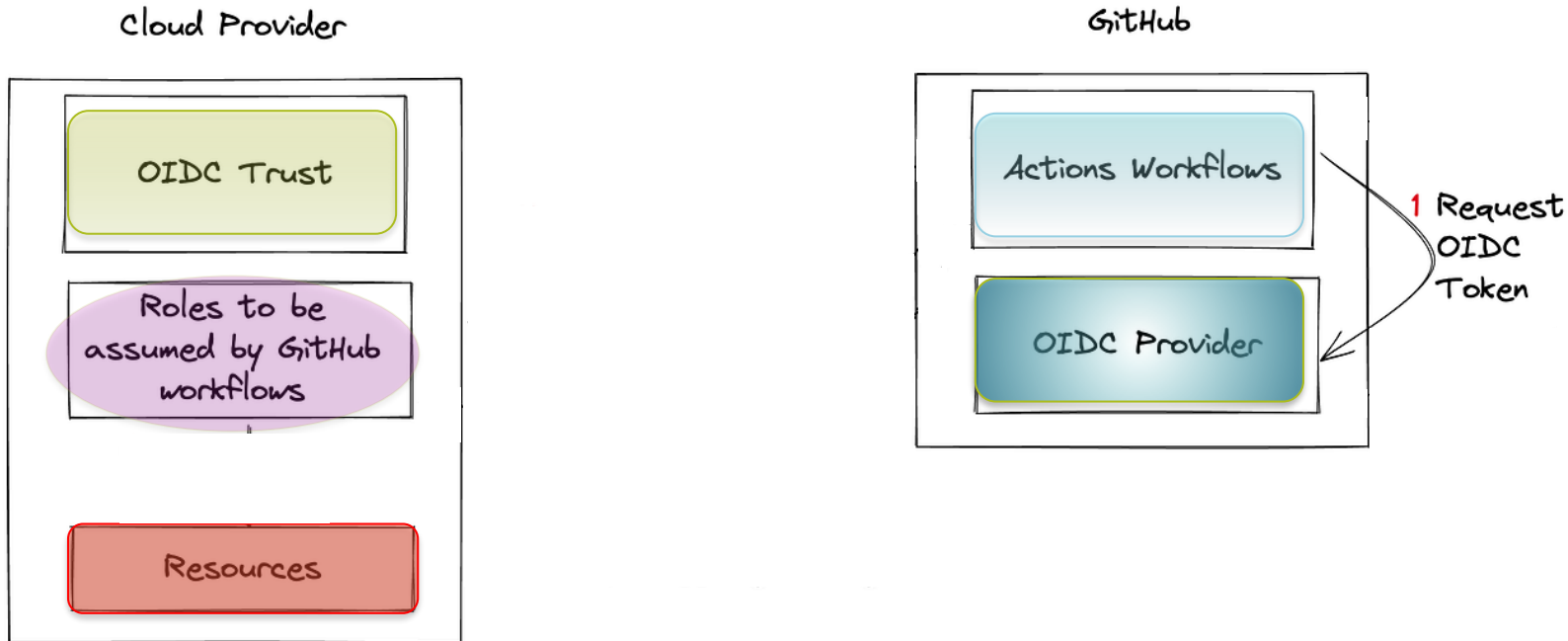
Cloud Provider



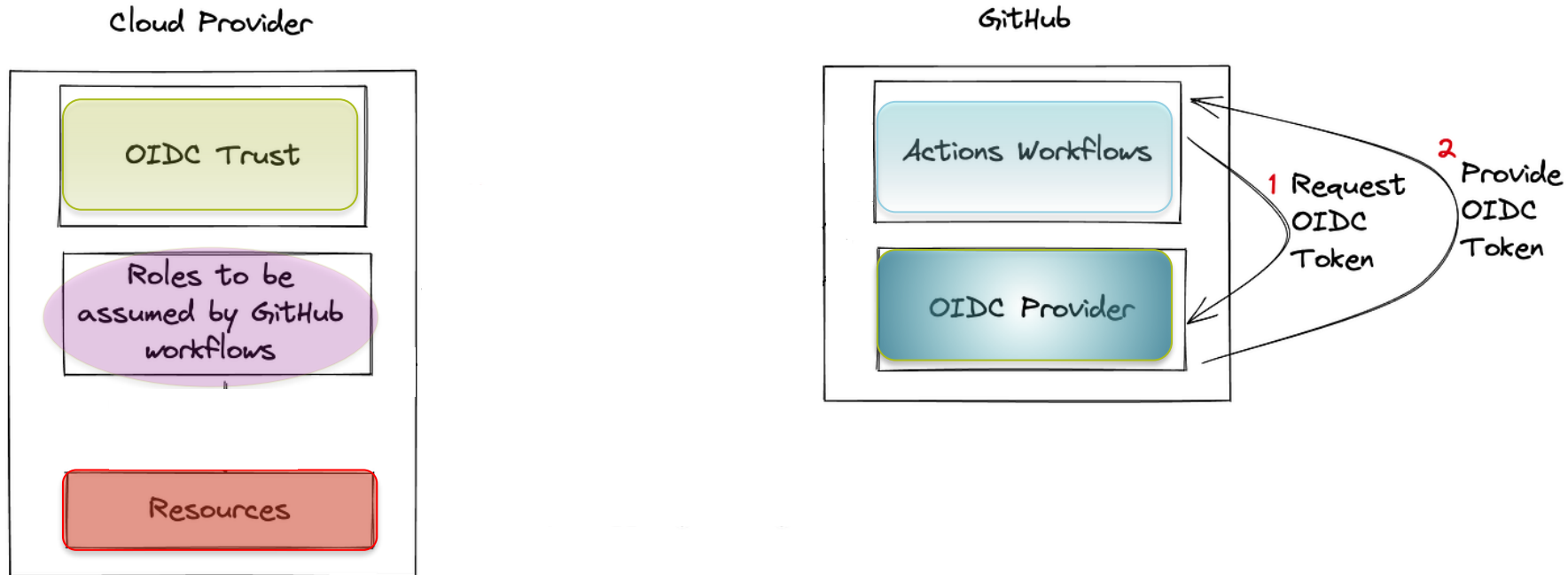
GitHub



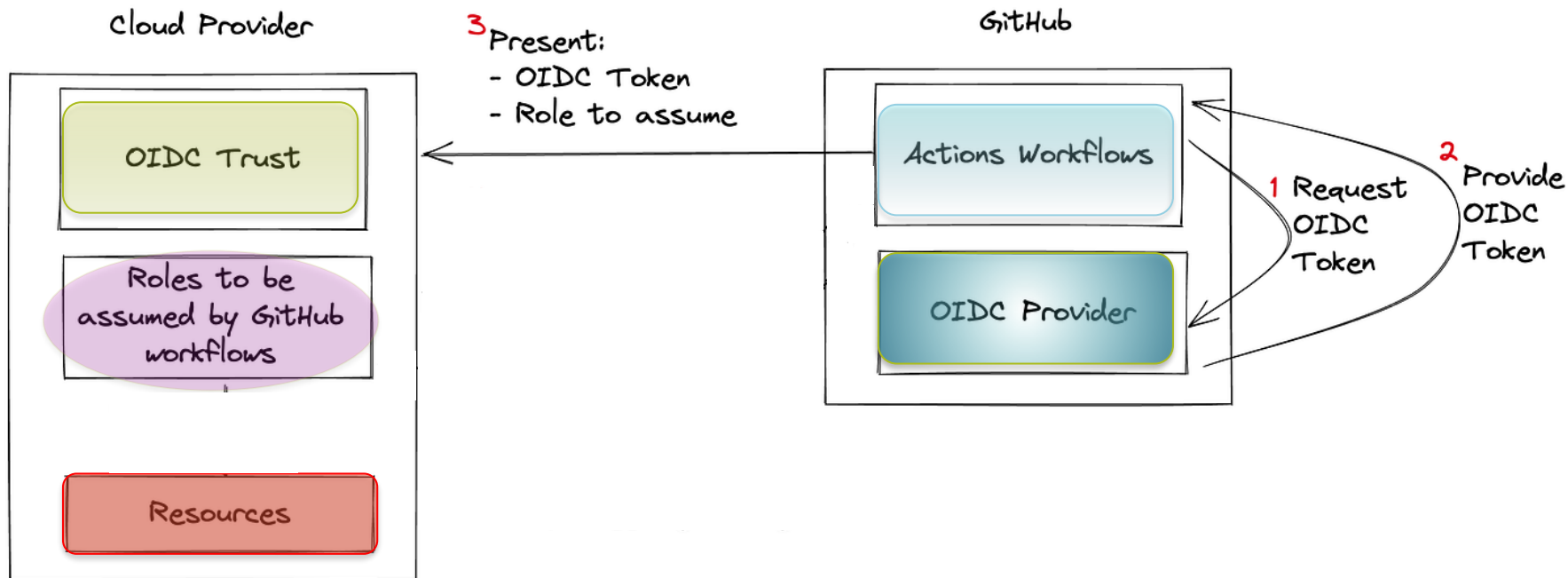
OIDC in GitHub Actions (Workflow)



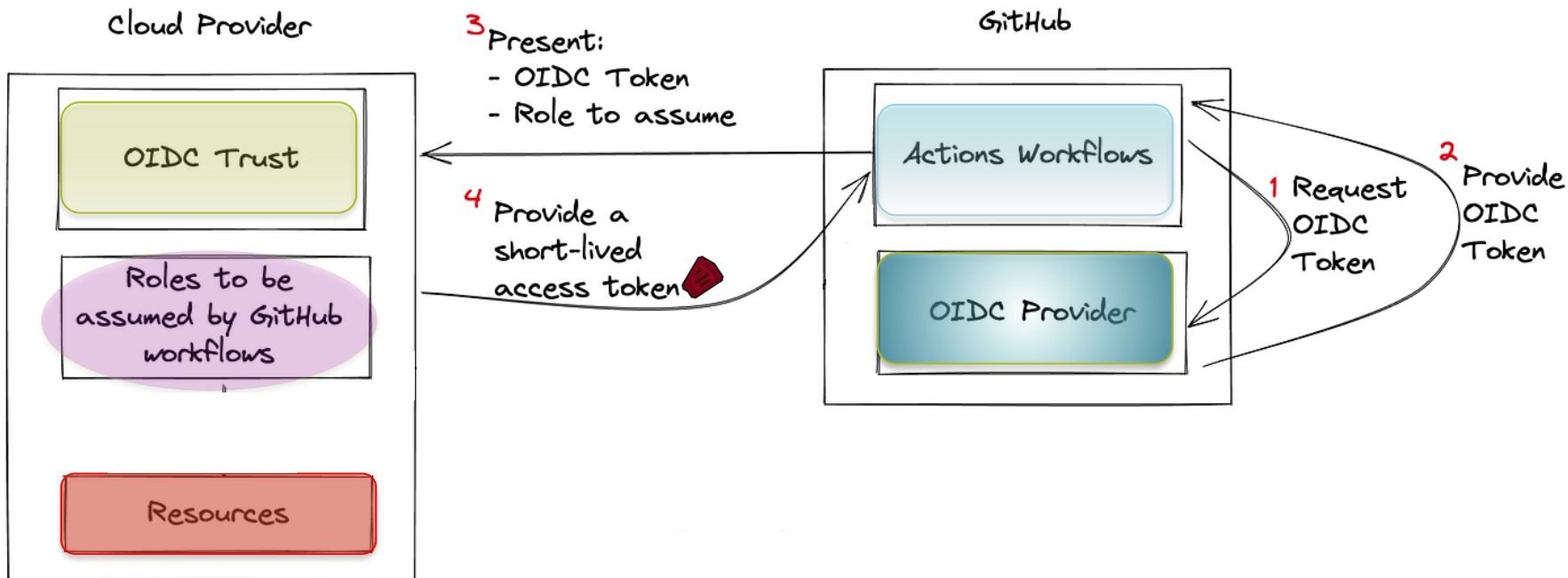
OIDC in GitHub Actions (Workflow)



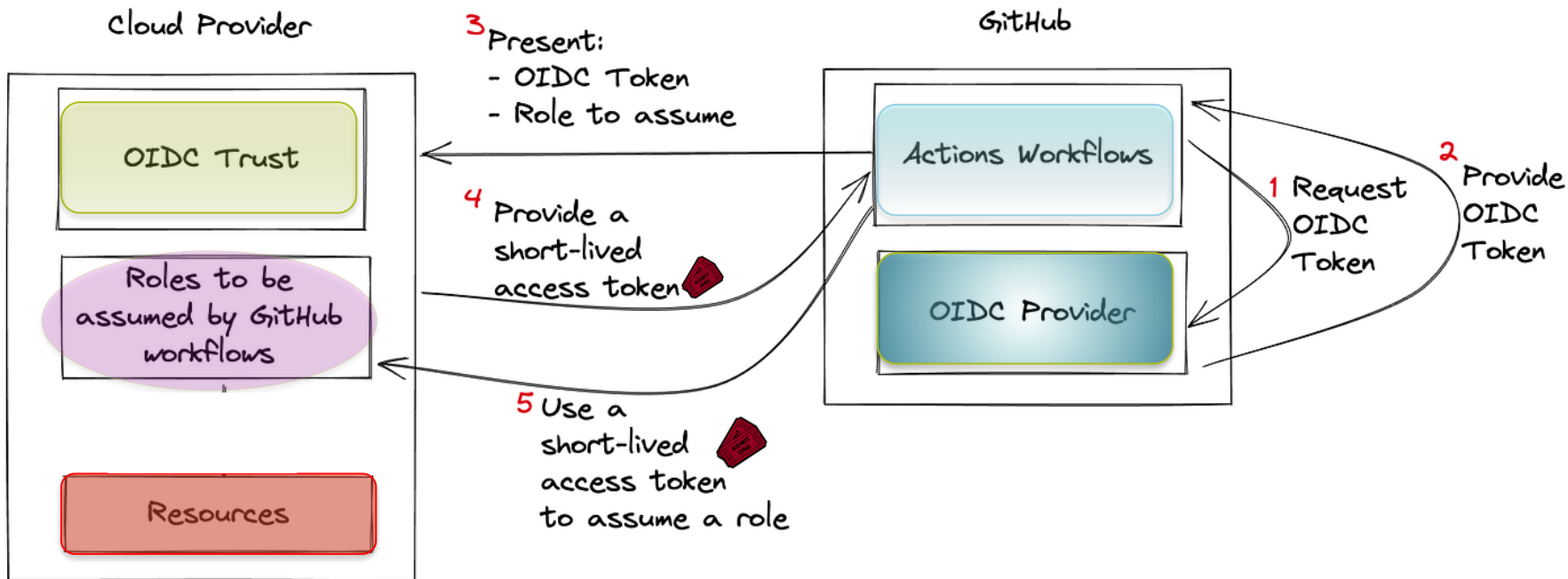
OIDC in GitHub Actions (Workflow)



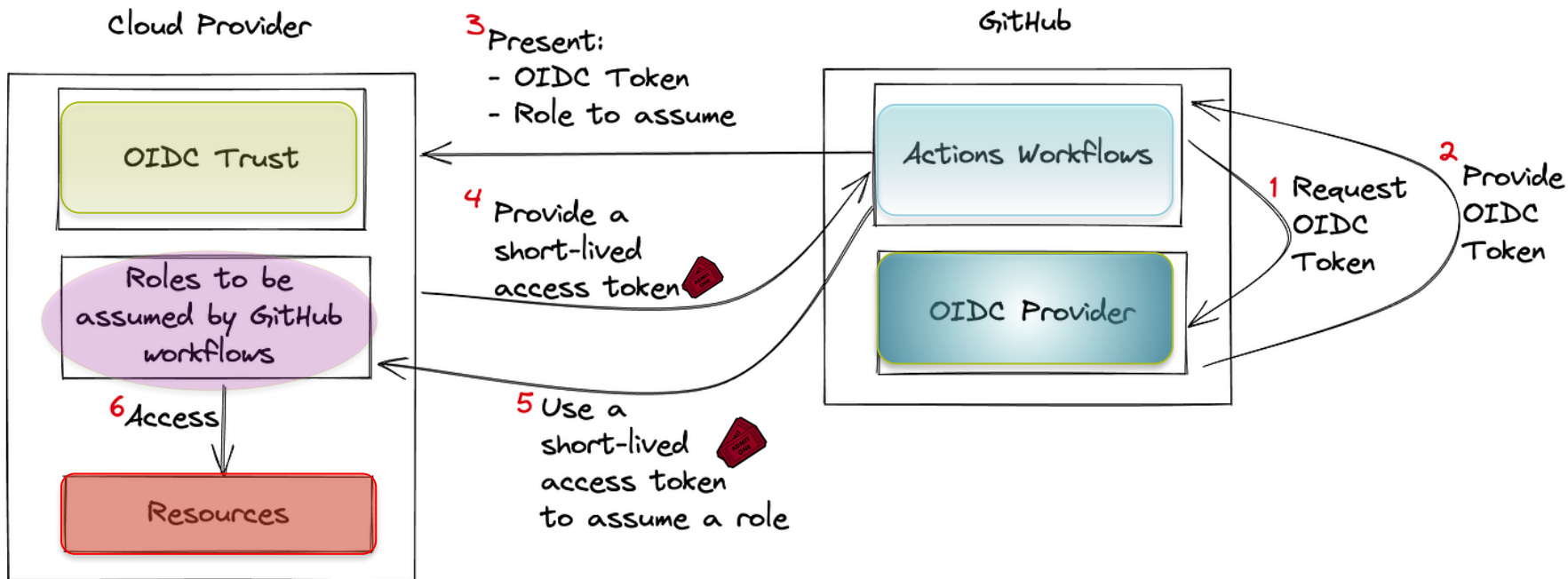
OIDC in GitHub Actions (Workflow)



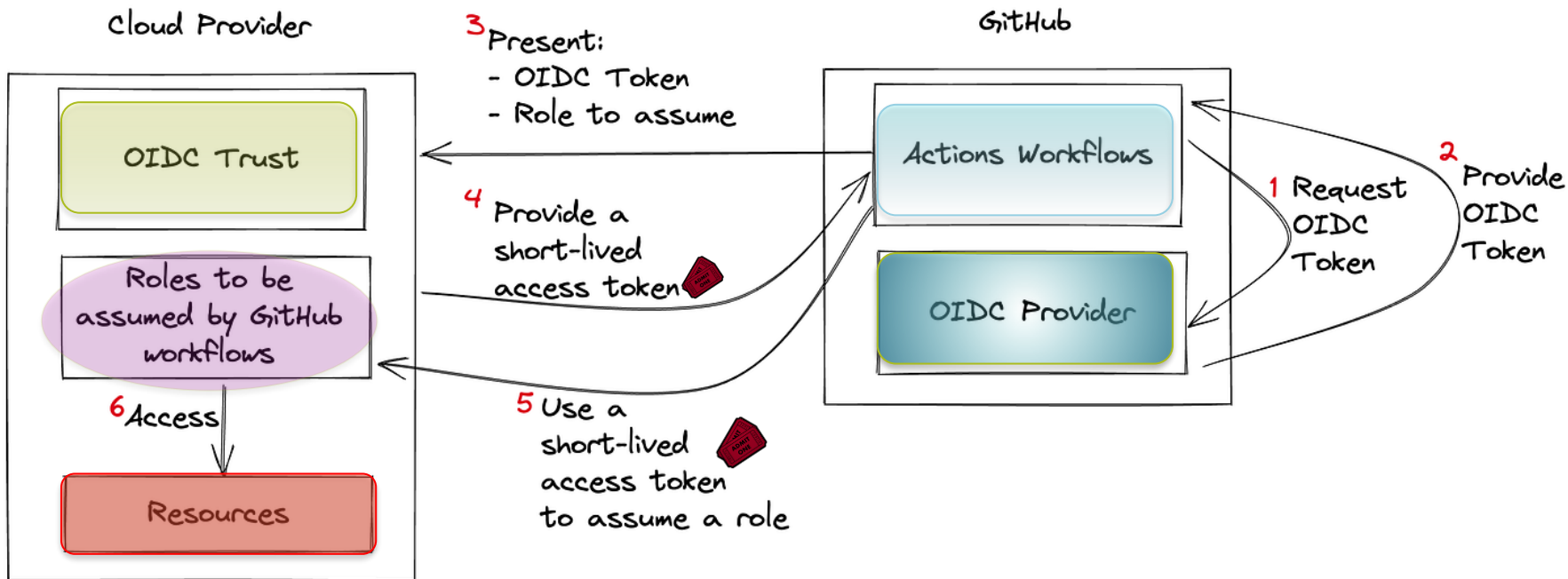
OIDC in GitHub Actions (Workflow)



OIDC in GitHub Actions (Workflow)



OIDC in GitHub Actions (Workflow)



Alibaba Cloud



IBM Cloud



ORACLE
CLOUD

GitHub actions workflow (JWT) token

- ❑ `actions-oidc-debugger` is the perfect tool to print your ID token (**JWT**) claims.

```
# Example: OIDC Token claims for "brokedba"s github actions workflow

{  "actor": "brokedba",
   "aud": "https://github.com/brokedba",
   "environment": "gcp-labs",
   "event_name": "push",
   "iss": "https://token.actions.githubusercontent.com",
   "job_workflow_ref": "brokedba/terraform-examples/.github/workflows/..",
   "ref": "refs/heads/git_actions",
   "ref_type": "branch",
   "repository": "brokedba/terraform-examples",
   "repository_owner": "brokedba",
   snip ...
   "sub": "repo:brokedba/terraform-examples:environment:gcp-labs",
   "workflow": "Terraform_gcp_vpc"
}
```

GitHub actions workflow (JWT) token

- ❑ `actions-oidc-debugger` is the perfect tool to print your ID token (**JWT**) claims.

```
# Example: OIDC Token claims for "brokedba"s github actions workflow

{
  "actor": "brokedba",
  "aud": "https://github.com/brokedba",
  "environment": "gcp-labs",
  "event_name": "push",
  "iss": "https://token.actions.githubusercontent.com",
  "job_workflow_ref": "brokedba/terraform-examples/.github/workflows/..",
  "ref": "refs/heads/git_actions",
  "ref_type": "branch",
  "repository": "brokedba/terraform-examples",
  "repository_owner": "brokedba",
  "sub": "repo:brokedba/terraform-examples:environment:gcp-labs",
  "workflow": "Terraform_gcp_vpc"
}
```

Audience

Issuer

GitHub actions workflow (JWT) token

- ❑ `actions-oidc-debugger` is the perfect tool to print your ID token (**JWT**) claims.

```
# Example: OIDC Token claims for "brokedba"s github actions workflow

{
  "actor": "brokedba",
  "aud": "https://github.com/brokedba",
  "environment": "gcp-labs",
  "event_name": "push",
  "iss": "https://token.actions.githubusercontent.com",
  "job_workflow_ref": "brokedba/terraform-examples/.github/workflows/..",
  "ref": "refs/heads/git_actions",
  "ref_type": "branch",
  "repository": "brokedba/terraform-examples",
  "repository_owner": "brokedba",
  "sub": "repo:brokedba/terraform-examples:environment:gcp-labs",
  "workflow": "Terraform_gcp_vpc"
}
```

Audience → `"aud": "https://github.com/brokedba",`

Issuer → `"iss": "https://token.actions.githubusercontent.com",`

Repo Ref → `"ref": "refs/heads/git_actions",`
`"ref_type": "branch",`
`"repository": "brokedba/terraform-examples",`
`"repository_owner": "brokedba",`

`snip ...`
`"sub": "repo:brokedba/terraform-examples:environment:gcp-labs",`
`"workflow": "Terraform_gcp_vpc"`

GitHub actions workflow (JWT) token

- ❑ `actions-oidc-debugger` is the perfect tool to print your ID token (**JWT**) claims.

```
# Example: OIDC Token claims for "brokedba"s github actions workflow

{
  "actor": "brokedba",
  "aud": "https://github.com/brokedba",
  "environment": "gcp-labs",
  "event_name": "push",
  "iss": "https://token.actions.githubusercontent.com",
  "job_workflow_ref": "brokedba/terraform-examples/.github/workflows/..",
  "ref": "refs/heads/git_actions",
  "ref_type": "branch",
  "repository": "brokedba/terraform-examples",
  "repository_owner": "brokedba",
  "sub": "repo:brokedba/terraform-examples:environment:gcp-labs",
  "workflow": "Terraform_gcp_vpc"
}
```

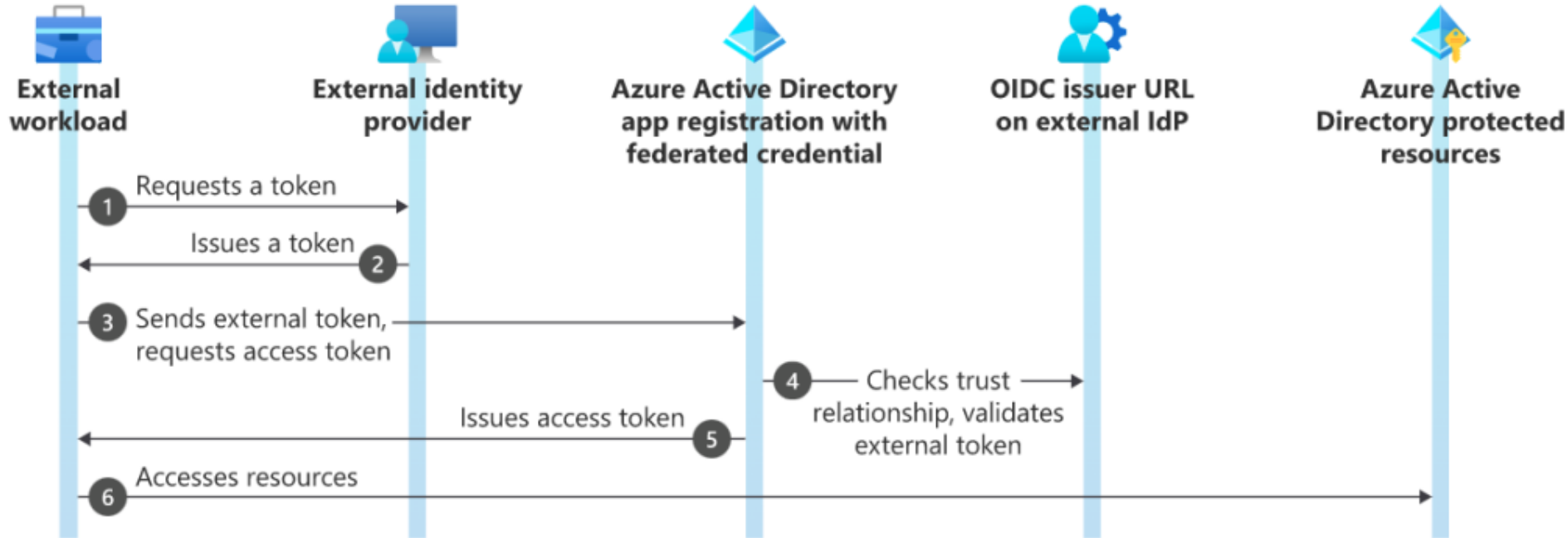
Audience → `"aud": "https://github.com/brokedba",`

Issuer → `"iss": "https://token.actions.githubusercontent.com",`

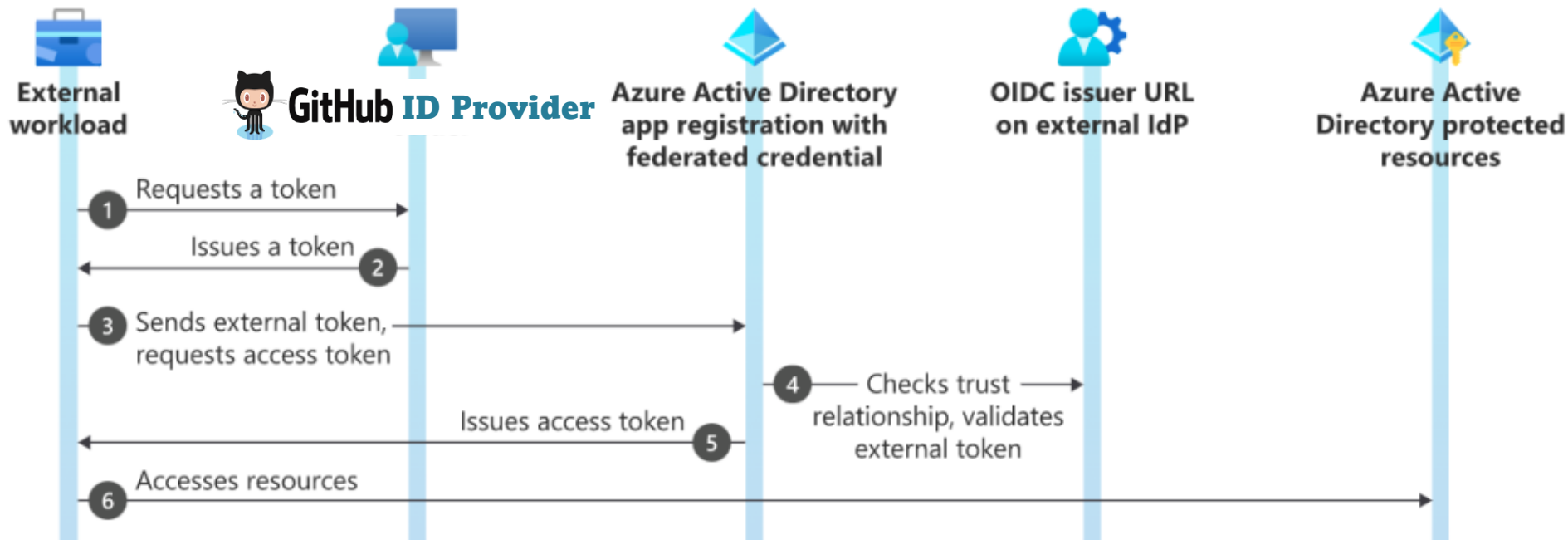
Repo Ref → `"ref": "refs/heads/git_actions",`
`"ref_type": "branch",`
`"repository": "brokedba/terraform-examples",`
`"repository_owner": "brokedba",`

Subject → `"sub": "repo:brokedba/terraform-examples:environment:gcp-labs",`
`"workflow": "Terraform_gcp_vpc"`

Cloud keyless access examples (Azure)



Cloud keyless access examples (Azure)



Cloud keyless access examples (Azure)

1. Workflow requests ID token from **GitHub ID Provider**
2. GitHub ID provider returns JWT **ID token**
3. **.Login** action in the workflow sends ID token to **Azure AD** requesting access
4. Azure AD verifies **claims** against Federated Workload Identity info
5. AZ AD sends an **Access Token** to the GitHub workflow.
6. The Git Actions workflow access Azure resources using the **Access Token**

Azure OIDC login Action

➤ Login action: Azure/login 

Input parameters

- ❑ **Client-id** of the principal associated with the federated Identity
- ❑ **tenant-id** of the principal associated with the federated Identity
- ❑ **subscription-id**

Prerequisites

- Setup **OIDC trust** in the cloud provider
- Set `permissions`: with `id-token: write` at job level

Azure OIDC login Action

➤ Login action: Azure/login

```
# Authenticate with Azure using OIDC Workload Federated Identity
- name: 'Az CLI login'
  uses: azure/login@v1
  with:
    client-id: ${ secrets.AZURE_CLIENT_ID }
    tenant-id: ${ secrets.AZURE_TENANT_ID }
    subscription-id: ${ secrets.AZURE_SUBSCRIPTION_ID }
```

Prerequisites

- Setup **OIDC trust** in the cloud provider
- Set `permissions`: with `id-token: write` at job level

Azure OIDC login Action

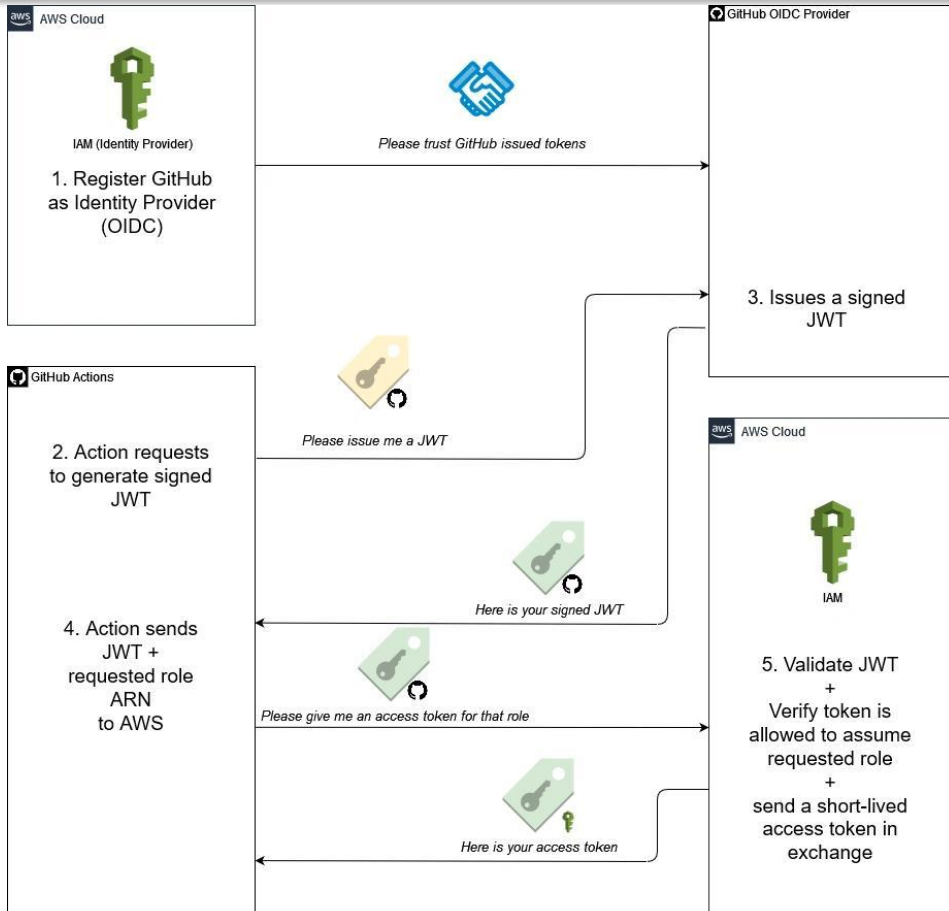
➤ Login action: Azure/login

```
# Authenticate with Azure using OIDC Workload Federated Identity
- name: 'Az CLI login'
  uses: azure/login@v1
  with:
    client-id: ${ secrets.AZURE_CLIENT_ID }
    tenant-id: ${ secrets.AZURE_TENANT_ID }
    subscription-id: ${ secrets.AZURE_SUBSCRIPTION_ID }
```

Prerequisites

- Setup **OIDC trust** in the cloud provider
- Set `permissions`: with `id-token: write` at job level

Cloud keyless access examples ()



Cloud keyless access examples ()

1. Set up **OIDC trust** between AWS role and GitHub ID token of your workflow.
2. [configure-aws-credentials](#) requests a signed **JWT** token during workflow run.
3. GitHub ID provider issues a signed **JWT** to the GitHub actions workflow
4. The action sends the **JWT** and the requested role to AWS.
5. AWS validates the **JWT claims** and requested role and sends back an **Access Token**
6. The Git Actions workflow access AWS resources using the **Access Token**

aws OIDC login Action

➤ Login action: [aws-actions/configure-aws-credentials](https://github.com/aws-actions/configure-aws-credentials) 

Input parameters

- ❑ **role-to-assume** of the Web Identity `arn:aws:iam::123456789100:role/my-github-actions-role`
- ❑ **role-session-name** default name is GitHubActions
- ❑ **aws-region**

Prerequisites

- Setup **OIDC trust** in the cloud provider
- Set `permissions`: with `id-token: write` at job level

aws OIDC login Action

➤ Login action: [aws-actions/configure-aws-credentials](https://github.com/aws-actions/configure-aws-credentials) 

```
# Authenticate with AWS using OIDC Workload Federated Identity
- name: 'Configure AWS credentials'
  uses: aws-actions/configure-aws-credentials@v2
  with:
    role-to-assume: ${ secrets.AWS_ROLE_TO_ASSUME }
    role-session-name-id: ${ vars.My_sessionName }
    aws-region: ${ vars.AWS_REGION }
```

Prerequisites

- Setup **OIDC trust** in the cloud provider
- Set `permissions`: with `id-token: write` at job level

aws OIDC login Action

➤ Login action: [aws-actions/configure-aws-credentials](https://aws.amazon.com/buildershub/docs/configure-aws-credentials/) 

```
# Authenticate with AWS using OIDC Workload Federated Identity
- name: 'Configure AWS credentials'
  uses: aws-actions/configure-aws-credentials@v2
  with:
    role-to-assume: ${{ secrets.AWS_ROLE_TO_ASSUME }}
    role-session-name-id: ${{ vars.My_sessionName }}
    aws-region: ${{ vars.AWS_REGION }}
```

Prerequisites

- Setup **OIDC trust** in the cloud provider
- Set `permissions`: with `id-token: write` at job level

OIDC login Action

➤ Login action: [google-github-actions/auth](#)



Input parameters

- ❑ **workload-identity-provider** Full ID of Workload IdP
- ❑ **service-account** Email address of the GCP service Account

Prerequisites

- Setup **OIDC trust** in the cloud provider
- Set `permissions`: with `id-token: write` at job level

OIDC login Action

➤ Login action: [google-github-actions/auth](#)



Input parameters

❑ **workload-identity-provider** Full ID of Workload IdP

```
projects/123456789/locations/global/workloadIdentityPools/github-pool/providers/my-provider
```

❑ **service-account** Email address of the GCP service Account

Prerequisites

- Setup **OIDC trust** in the cloud provider
- Set `permissions`: with `id-token: write` at job level

OIDC login Action

➤ Login action: [google-github-actions/auth](https://github.com/google-github-actions/auth)



```
# Authenticate with GCP using OIDC Workload Federated Identity
- name: 'Authenticate to Google Cloud'
  uses: google-github-actions/auth@v1
  with:
    workload_identity_provider: ${{ secrets.WORKLOAD_ID_PROVIDER }}
    service-account: ${{ secrets.SERVICE_ACCOUNT }}
    token_format: 'access_token'
```

Prerequisites

- Setup **OIDC trust** in the cloud provider
- Set `permissions: write id-token:` at job level

OIDC login Action

➤ Login action: [google-github-actions/auth](https://github.com/google-github-actions/auth)



```
# Authenticate with GCP using OIDC Workload Federated Identity
- name: 'Authenticate to Google Cloud'
  uses: google-github-actions/auth@v1
  with:
    workload_identity_provider: '${{ secrets.WORKLOAD_ID_PROVIDER }}'
    service-account: '${{ secrets.SERVICE_ACCOUNT }}'
    token_format: 'access_token'
```

Prerequisites

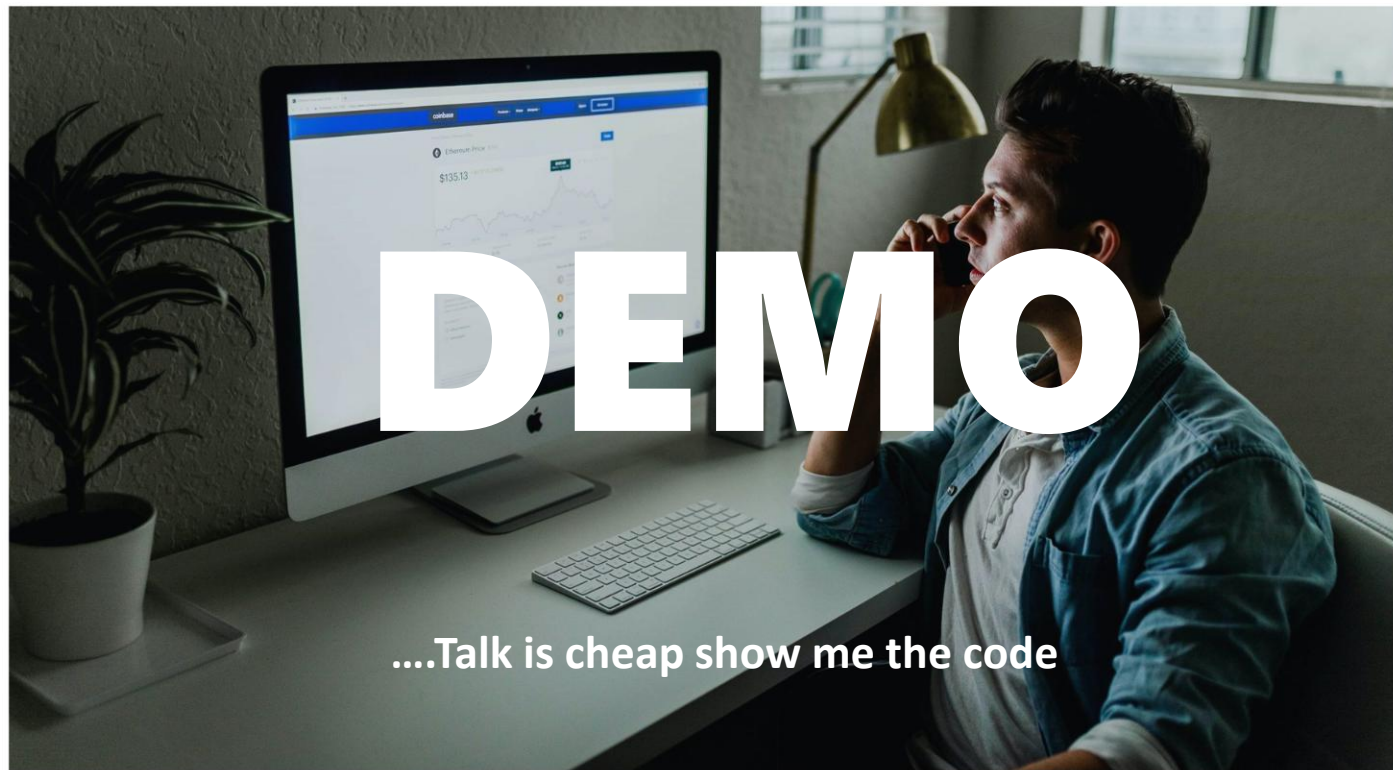
- Setup **OIDC trust** in the cloud provider
- Set `permissions: write id-token:` at job level

Github Deploy in AWS using OIDC

Blog post version



SCAN ME



....Talk is cheap show me the code

OIDC in GitHub actions



THANK YOU



`Tech Beats unplugged`

**Tech Beats
Unplugged**



Kosseila Hd

Solutions architect @CloudThrill

Cloud Dude



CloudThrill.io



Twitter.com/CloudDude_



Linkedin.com/in/kousshd



GitHub.com/brokedba



SCAN ME

CloudThrill
expertise worth sharing...